

Low-Level Design (LLD) Interview Guide

Object-oriented design problems with Java 8+ — patterns, concurrency, and error handling

Target level: Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

What LLD interviews actually test

An LLD (also called "object-oriented design" or "machine coding") round hands you a real-world thing — *"design a parking lot"*, *"design an elevator"* — and watches **how you model it in classes**. Unlike a DSA round, there is rarely a single correct answer. You are judged on:

- **Clarifying requirements** before you write anything — scope the problem, don't assume.
- **Identifying the right entities** (the nouns) and their **relationships** (has-a, is-a).
- **Applying SOLID principles** so the design is extensible, not a god-class.
- **Choosing the right design pattern** — and being able to say *why*.
- **Handling concurrency** where multiple actors touch shared state (seat booking, ATM).
- **Handling errors** gracefully with meaningful exceptions, not silent failures.
- **Walking a use case** end-to-end to prove the design works.

This guide gives each problem as **class skeletons + key method bodies** — the same balance you'd write on a whiteboard: enough structure and core logic to prove the design, without drowning in boilerplate getters.

The 5-step method (use this on every problem)

1. CLARIFY → Ask about scope. (How many floors? Payment types? Single or multiple elevators? Real-time or batch?)
 2. ENTITIES → List the nouns. Each becomes a class or enum. (ParkingLot, Level, Spot, Vehicle, Ticket, ...)
 3. RELATE → has-a (composition) vs is-a (inheritance). Draw the class diagram.
 4. BEHAVE → Assign responsibilities (methods) to classes. Apply SOLID + a design pattern where it fits.
 5. VERIFY → Walk one full use case. Add concurrency + error handling where shared state or failure is possible.
-

The design-pattern toolkit

These are the patterns that cover ~90% of LLD problems. Each problem in this guide names which it uses and why.

Pattern	One-line intent	Shows up in
Strategy	Swap an algorithm at runtime behind an interface	Parking-fee calc, cache eviction, pricing
State	Object changes behavior when its internal state changes	Vending machine, ATM, elevator
Factory	Create objects without naming the concrete class	Vehicle creation, notification channels
Singleton	Exactly one shared instance	ParkingLot controller, logger, config
Observer	Notify many subscribers when something changes	Notifications, pub-sub, seat availability
Chain of Responsibility	Pass a request along a chain of handlers	Logger levels, ATM cash dispensing
Builder	Construct a complex object step by step	Tickets, orders with many optional fields
Command	Encapsulate a request as an object	Elevator requests, undo/redo

SOLID in one breath

- **S**ingle Responsibility — one class, one reason to change.
- **O**pen/Closed — open to extension, closed to modification (add a new `Strategy`, don't edit a `switch`).
- **L**iskov Substitution — subtypes must be usable through the base type.
- **I**nterface Segregation — many small interfaces beat one fat one.
- **D**ependency Inversion — depend on abstractions (`interface`), not concretions.

A note on concurrency

Where multiple actors mutate shared state (two users booking the same seat, two threads dispensing cash), this guide shows the safe approach: `synchronized` blocks, `ReentrantLock`, `AtomicInteger`, or concurrent collections (`ConcurrentHashMap`), and explains the race being prevented. The rule of thumb: **lock the smallest critical section that keeps the invariant true.**

The 13 designs

#	Design	Headline pattern(s)	Concurrency?
1	Parking Lot	Strategy, Factory, Singleton	Spot allocation

#	Design	Headline pattern(s)	Concurrency?
2	Elevator System	State, Command	Request dispatch
3	Library Management	(entity modeling), Strategy for fines	Low
4	Vending Machine	State	Inventory
5	BookMyShow (tickets)	Singleton, Observer	Seat locking (high)
6	Splitwise	Strategy (split types)	Balance updates
7	Tic-Tac-Toe	State / Strategy for win-check	None
8	ATM	State, Chain of Responsibility	Cash dispensing
9	Logging Framework	Chain of Responsibility	Thread-safe writes
10	Rate Limiter	Strategy (token bucket / window)	High (atomic counters)
11	Notification Service	Observer, Strategy, Factory	Async dispatch
12	Pluggable Cache (LRU/LFU)	Strategy	Thread-safe ops
13	Ride Hailing / Food Delivery (mini)	Strategy (matching, pricing), Observer	Matching

1. Parking Lot

The problem in one line. Design a parking lot that can park and un-park vehicles of different sizes across multiple levels, issue tickets, and charge a fee based on how long a vehicle stayed.

Step 1 — Clarify requirements

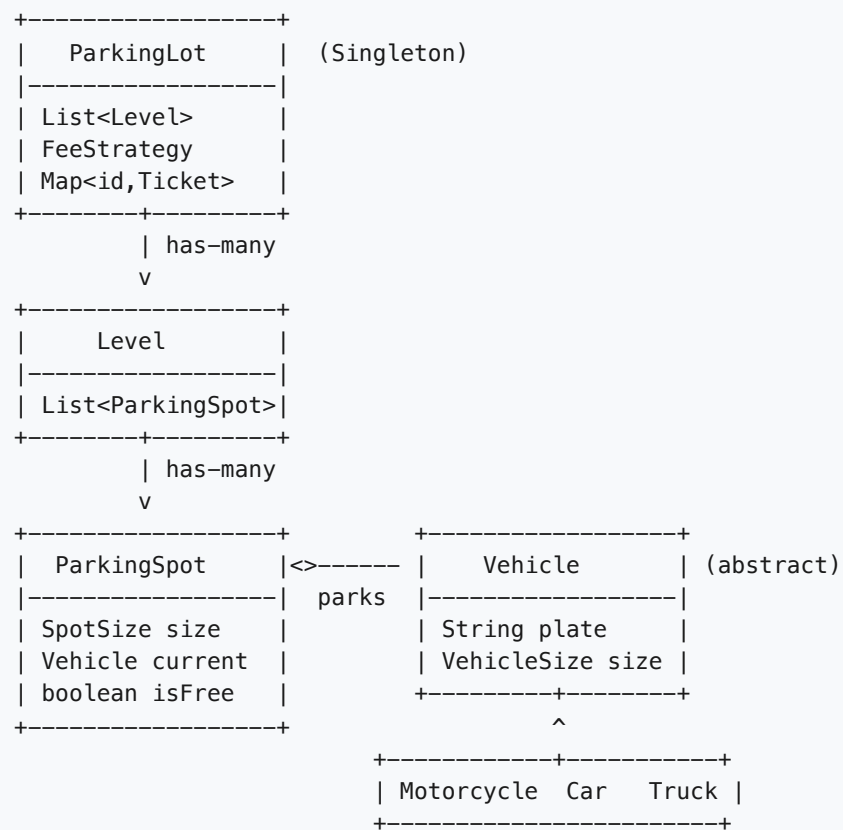
Questions worth asking out loud (and the assumptions this design makes):

- **Vehicle types?** Motorcycle, Car, Bus/Truck → three spot sizes (SMALL, MEDIUM, LARGE).
- **Multiple levels/floors?** Yes — the lot has several `Level`s, each with many spots.
- **Fit rules?** A vehicle fits in a spot of equal-or-larger size (a car can take a large spot if no medium is free). *Confirm this — some variants forbid it.*
- **Payment?** Fee = hourly rate × hours parked. Pluggable so pricing can change.
- **Concurrency?** Multiple entry gates may request a spot at the same instant — two cars must never be assigned the **same** spot.

Step 2 — Entities (the nouns)

`ParkingLot`, `Level`, `ParkingSpot`, `Vehicle` (+ subtypes), `Ticket`, `FeeStrategy`.

Step 3 — Relationships (class diagram)



`FeeStrategy` (interface) <|-- `HourlyFeeStrategy` [Strategy pattern]

Step 4 — Patterns chosen (and why)

- **Singleton** for `ParkingLot` — there is exactly one lot controller; everything routes through it. (In production you'd inject it instead, but Singleton is the expected LLD answer.)
- **Strategy** for `FeeStrategy` — pricing rules change (hourly, flat, weekend surge) without touching the lot. Open/Closed in action.
- **Factory** for creating `Vehicle`s from a type string — callers don't `new` concrete classes.
- **Inheritance** for `Vehicle` subtypes; **composition** everywhere else (a lot *has* levels, a level *has* spots).

Step 5 — Skeleton + key methods

Enums and Vehicle hierarchy

```
enum VehicleSize { SMALL, MEDIUM, LARGE }

abstract class Vehicle {
    private final String plate;
    private final VehicleSize size;
    Vehicle(String plate, VehicleSize size) { this.plate = plate; this.size = size; }
    public VehicleSize getSize() { return size; }
    public String getPlate() { return plate; }
}

class Motorcycle extends Vehicle { Motorcycle(String p) { super(p, VehicleSize.SMALL); } }
class Car extends Vehicle { Car(String p) { super(p, VehicleSize.MEDIUM); } }
class Truck extends Vehicle { Truck(String p) { super(p, VehicleSize.LARGE); } }

// Factory: create a vehicle without the caller naming the concrete class.
class VehicleFactory {
    static Vehicle create(String type, String plate) {
        switch (type.toLowerCase()) {
            case "motorcycle": return new Motorcycle(plate);
            case "car": return new Car(plate);
            case "truck": return new Truck(plate);
            default: throw new IllegalArgumentException("Unknown vehicle type: " + type);
        }
    }
}
```

Spot — the unit of shared state

```
class ParkingSpot {
    private final String id;
    private final VehicleSize size;
    private Vehicle vehicle; // null when free

    ParkingSpot(String id, VehicleSize size) { this.id = id; this.size = size; }

    boolean isFree() { return vehicle == null; }

    // A vehicle fits a spot of equal or larger size.
    boolean canFit(Vehicle v) { return isFree() && v.getSize().ordinal() <= size.ordinal(); }

    void assign(Vehicle v) { this.vehicle = v; }
    void release() { this.vehicle = null; }
    String getId() { return id; }
}
```

Fee strategy (Strategy pattern)

```
interface FeeStrategy {
    double calculate(Ticket ticket, long exitEpochSec);
}

class HourlyFeeStrategy implements FeeStrategy {
    private final double ratePerHour;
    HourlyFeeStrategy(double ratePerHour) { this.ratePerHour = ratePerHour; }

    public double calculate(Ticket ticket, long exitEpochSec) {
        long seconds = Math.max(0, exitEpochSec - ticket.getEntryEpochSec());
        long hours = (long) Math.ceil(seconds / 3600.0); // round up partial hours
        return Math.max(1, hours) * ratePerHour; // minimum one hour
    }
}
```

Ticket

```
class Ticket {
    private final String id;
    private final Vehicle vehicle;
    private final ParkingSpot spot;
    private final long entryEpochSec;

    Ticket(String id, Vehicle v, ParkingSpot spot, long entryEpochSec) {
        this.id = id; this.vehicle = v; this.spot = spot; this.entryEpochSec = entryEpochSec;
    }
    String getId() { return id; }
    ParkingSpot getSpot() { return spot; }
    long getEntryEpochSec() { return entryEpochSec; }
}
```

The controller — with concurrency + error handling

```
class ParkingLot {
    private static final ParkingLot INSTANCE = new ParkingLot();    // Singleton
    public static ParkingLot getInstance() { return INSTANCE; }

    private final List<Level> levels = new ArrayList<>();
    private final Map<String, Ticket> activeTickets = new ConcurrentHashMap<>();
    private FeeStrategy feeStrategy = new HourlyFeeStrategy(10.0);
    private final AtomicLong ticketSeq = new AtomicLong(0);

    private ParkingLot() { }

    public void addLevel(Level level) { levels.add(level); }
    public void setFeeStrategy(FeeStrategy s) { this.feeStrategy = s; }    // swap pricing at

    // Park a vehicle. Returns a ticket, or throws if the lot is full.
    // synchronized so two gates can't grab the same spot (the critical race).
    public synchronized Ticket park(Vehicle vehicle, long nowEpochSec) {
        ParkingSpot spot = findSpot(vehicle);
        if (spot == null) {
            throw new IllegalStateException("Parking full for size " + vehicle.getSize());
        }
        spot.assign(vehicle);
        Ticket ticket = new Ticket("T" + ticketSeq.incrementAndGet(), vehicle, spot, nowEpochSec);
        activeTickets.put(ticket.getId(), ticket);
        return ticket;
    }

    // Un-park: free the spot, compute the fee, close the ticket.
    public synchronized double unpark(String ticketId, long exitEpochSec) {
        Ticket ticket = activeTickets.remove(ticketId);
        if (ticket == null) {
            throw new NoSuchElementException("Invalid or already-used ticket: " + ticketId);
        }
        ticket.getSpot().release();
        return feeStrategy.calculate(ticket, exitEpochSec);
    }

    private ParkingSpot findSpot(Vehicle v) {
        for (Level level : levels) {
            ParkingSpot spot = level.findAvailable(v);
            if (spot != null) return spot;
        }
        return null;
    }
}

class Level {
    private final List<ParkingSpot> spots;
    Level(List<ParkingSpot> spots) { this.spots = spots; }

    ParkingSpot findAvailable(Vehicle v) {
        return spots.stream().filter(s -> s.canFit(v)).findFirst().orElse(null);
    }
}
```

Why `synchronized on park / unpark` ? The check-then-assign on a spot is a classic race: two threads could both see the same spot free and both assign it. Serializing the allocation keeps the invariant "one

spot ↔ at most one vehicle." For a very large lot you'd lock per-level (finer-grained) instead of the whole lot, but whole-lot locking is the correct, simple answer to state first.

Use-case walkthrough

```
1. Vehicle car = VehicleFactory.create("car", "KA-01-1234");
2. Ticket t = ParkingLot.getInstance().park(car, now);
   -> findSpot scans levels, finds a free MEDIUM (or LARGE) spot
   -> spot.assign(car); ticket T1 stored in activeTickets
3. ... 2.5 hours later ...
4. double fee = ParkingLot.getInstance().unpark(t.getId(), later);
   -> spot released; HourlyFeeStrategy: ceil(2.5) = 3 hours * 10 = 30.0
5. Second car arrives when lot is full -> park() throws IllegalStateException
   -> caller shows "Lot Full" at the gate.
```

Extensions an interviewer may probe

- **Multiple entrances with a display board** → add an `Observer` so gates see live counts.
 - **Reserved/EV spots** → new `SpotType`; `canFit` becomes a `Strategy`.
 - **Surge pricing** → drop in a new `FeeStrategy`; nothing else changes (Open/Closed).
-

2. Elevator System

The problem in one line. Design a multi-elevator system for a building where a central controller accepts hallway calls and in-car floor requests, and dispatches each request to the best-suited elevator.

Step 1 — Clarify requirements

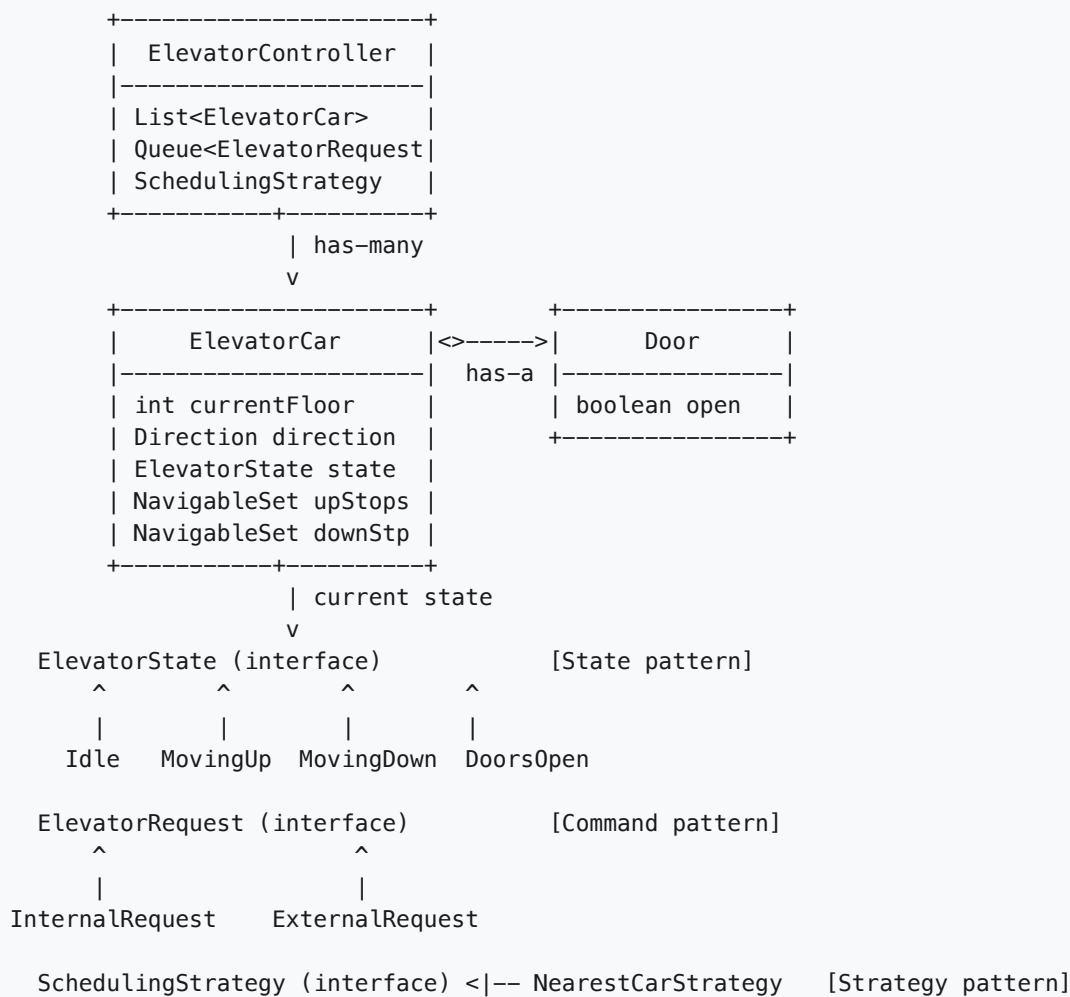
Questions worth asking out loud (and the assumptions this design makes):

- **How many elevators/floors?** Multiple elevators (a "bank"), serving a fixed floor range `[minFloor, maxFloor]` shared by the building.
- **Two kinds of requests?** Yes — an **external** request (hallway UP/DOWN button on a floor) and an **internal** request (a floor button pressed inside the car). They're handled differently: external needs a car *assigned*; internal targets a car that already has a passenger.
- **Dispatch strategy?** Pick the "best" elevator for an external call — nearest car already heading that way beats an idle car beats a car moving the wrong direction. (*Confirm — some variants want capacity/weight limits too; out of scope here.*)
- **Concurrency?** Hallway buttons on every floor and every car's panel can fire at the same instant — the controller must accept concurrent requests without corrupting a car's stop list or double-assigning work.
- **Capacity/weight limits?** Out of scope — assume unlimited capacity per car.
- **Invalid input?** Requesting a floor outside `[minFloor, maxFloor]` is a caller error and should fail loudly, not be silently ignored.

Step 2 — Entities (the nouns)

`ElevatorController` (dispatcher), `ElevatorCar`, `ElevatorState` (+ concrete states),
`ElevatorRequest` (+ `InternalRequest` / `ExternalRequest`), `SchedulingStrategy`, `Door`.

Step 3 — Relationships (class diagram)



Step 4 — Patterns chosen (and why)

- **State** for **ElevatorState** — a car's behavior (where it moves next, whether doors open) depends entirely on *which state it's in*; this replaces a tangle of `if (direction == UP && ...)` checks with one class per state.
- **Command** for **ElevatorRequest** — a button press (hallway or in-car) is encapsulated as an object that can be queued, logged, or replayed, and the button itself doesn't need to know *how* the request is fulfilled, only that it can `execute()`.
- **Strategy** for **SchedulingStrategy** — the "which car should answer this call" rule is exactly the kind of policy that changes (nearest-car today, load-balanced or zone-based tomorrow) without touching the controller. Open/Closed in action.
- **Composition** everywhere else — a controller *has* cars, a car *has* a door and a state; no inheritance needed beyond the two small interface hierarchies above.

Step 5 — Skeleton + key methods

Enums and Door

```
enum Direction { UP, DOWN, IDLE }

class Door {
    private volatile boolean open = false;
    void open() { open = true; }
    void close() { open = false; }
    boolean isOpen() { return open; }
}
```

ElevatorState — the State pattern

```
// Each concrete state decides what the car does on its next "tick" and what
// state comes after. The car itself never branches on direction/mode again.
interface ElevatorState {
    void handle(ElevatorCar car);
}

class IdleState implements ElevatorState {
    public void handle(ElevatorCar car) {
        // Woken up by a new stop being added; head toward whichever queue is non-empty.
        if (!car.getUpStops().isEmpty()) {
            car.setDirection(Direction.UP);
            car.setState(new MovingUpState());
        } else if (!car.getDownStops().isEmpty()) {
            car.setDirection(Direction.DOWN);
            car.setState(new MovingDownState());
        }
        // else: nothing queued – stay idle.
    }
}

class MovingUpState implements ElevatorState {
    public void handle(ElevatorCar car) {
        Integer next = car.getUpStops().pollFirst();    // nearest floor above, in order
        if (next == null) {
            car.setState(new IdleState());
            return;
        }
        car.moveTo(next);
        car.setState(new DoorsOpenState());            // arrived – open doors next tick
    }
}

class MovingDownState implements ElevatorState {
    public void handle(ElevatorCar car) {
        Integer next = car.getDownStops().pollLast();  // nearest floor below, in order
        if (next == null) {
            car.setState(new IdleState());
            return;
        }
        car.moveTo(next);
        car.setState(new DoorsOpenState());
    }
}

class DoorsOpenState implements ElevatorState {
    public void handle(ElevatorCar car) {
        car.getDoor().open();
        // ... dwell for a few seconds in a real system; simulation collapses that here ...
        car.getDoor().close();
        // Keep going the same direction only if there's more work that way;
        // otherwise flip direction, or go idle if the car is empty of requests.
        if (car.getDirection() == Direction.UP && !car.getUpStops().isEmpty()) {
            car.setState(new MovingUpState());
        } else if (car.getDirection() == Direction.DOWN && !car.getDownStops().isEmpty()) {
            car.setState(new MovingDownState());
        } else if (!car.getUpStops().isEmpty()) {
            car.setDirection(Direction.UP);
            car.setState(new MovingUpState());
        }
    }
}
```

```
    } else if (!car.getDownStops().isEmpty()) {  
        car.setDirection(Direction.DOWN);  
        car.setState(new MovingDownState());  
    } else {  
        car.setDirection(Direction.IDLE);  
        car.setState(new IdleState());  
    }  
}
```

ElevatorCar — the unit of shared state

```
class ElevatorCar {
    private final int id;
    private final int minFloor, maxFloor;
    private volatile int currentFloor;
    private volatile Direction direction = Direction.IDLE;
    private volatile ElevatorState state = new IdleState();

    // Concurrent, sorted stop sets: the controller thread adds external stops while the
    // car's own drive loop polls the nearest one – ConcurrentSkipListSet makes both safe
    // without an explicit lock, and keeps stops naturally ordered for pollFirst/pollLast.
    private final NavigableSet<Integer> upStops = new ConcurrentSkipListSet<>();
    private final NavigableSet<Integer> downStops = new ConcurrentSkipListSet<>(Comparator.re
    private final Door door = new Door();

    ElevatorCar(int id, int minFloor, int maxFloor) {
        this.id = id; this.minFloor = minFloor; this.maxFloor = maxFloor;
        this.currentFloor = minFloor;
    }

    // Record a new stop (from a hallway call assigned to this car, or a floor button
    // pressed inside it). Validates range, then wakes the car if it was idle.
    public void addStop(int floor) {
        if (floor < minFloor || floor > maxFloor) {
            throw new IllegalArgumentException(
                "Floor " + floor + " is out of range [" + minFloor + "," + maxFloor + "]");
        }
        if (floor > currentFloor) upStops.add(floor);
        else if (floor < currentFloor) downStops.add(floor);
        else door.open(); // already there – just open up

        if (state instanceof IdleState) state.handle(this); // kick the state machine awake
    }

    // One tick of the simulation/drive loop; delegates all logic to the current state.
    public void step() { state.handle(this); }

    int getCurrentFloor() { return currentFloor; }
    void moveTo(int floor) { this.currentFloor = floor; }
    Direction getDirection() { return direction; }
    void setDirection(Direction d) { this.direction = d; }
    ElevatorState getState() { return state; }
    void setState(ElevatorState s) { this.state = s; }
    NavigableSet<Integer> getUpStops() { return upStops; }
    NavigableSet<Integer> getDownStops() { return downStops; }
    Door getDoor() { return door; }
    int getId() { return id; }
    int getMinFloor() { return minFloor; }
    int getMaxFloor() { return maxFloor; }
}
```

ElevatorRequest — the Command pattern

```
interface ElevatorRequest {
    void execute();
}

// Someone inside the car pressed a floor button – the car is already known.
class InternalRequest implements ElevatorRequest {
    private final ElevatorCar car;
    private final int targetFloor;
    InternalRequest(ElevatorCar car, int targetFloor) { this.car = car; this.targetFloor = targetFloor; }
    public void execute() { car.addStop(targetFloor); }
}

// Someone on a floor pressed UP/DOWN – no car is known yet; the controller must pick one.
class ExternalRequest implements ElevatorRequest {
    private final ElevatorController controller;
    private final int floor;
    private final Direction direction;
    ExternalRequest(ElevatorController controller, int floor, Direction direction) {
        this.controller = controller; this.floor = floor; this.direction = direction;
    }
    public void execute() { controller.dispatch(floor, direction); }
}
```

SchedulingStrategy — the Strategy pattern

```
interface SchedulingStrategy {
    ElevatorCar selectCar(List<ElevatorCar> cars, int floor, Direction direction);
}

class NearestCarStrategy implements SchedulingStrategy {
    public ElevatorCar selectCar(List<ElevatorCar> cars, int floor, Direction direction) {
        ElevatorCar best = null;
        int bestCost = Integer.MAX_VALUE;
        for (ElevatorCar car : cars) {
            int cost = costOf(car, floor, direction);
            if (cost < bestCost) { bestCost = cost; best = car; }
        }
        return best;
    }

    // Lower cost = better fit. This is the simple LOOK/SCAN idea: a car already
    // moving toward the floor in the requested direction is "on the way" and wins;
    // an idle car is next-best; a car moving away has to finish its run first.
    private int costOf(ElevatorCar car, int floor, Direction dir) {
        int distance = Math.abs(car.getCurrentFloor() - floor);
        if (car.getDirection() == Direction.IDLE) return distance;

        boolean sameDirection = car.getDirection() == dir;
        boolean onTheWay = sameDirection &&
            (dir == Direction.UP ? car.getCurrentFloor() <= floor
            : car.getCurrentFloor() >= floor);
        if (onTheWay) return distance;
        return distance + 1_000; // heavy penalty: wrong-direction car, picks up later
    }
}
```


ElevatorController — dispatch, queueing, and the race

```
class ElevatorController {
    private final List<ElevatorCar> cars;
    private final SchedulingStrategy scheduler;

    // Every hallway button and every in-car button funnels a Command in here from
    // whatever thread handled the press – LinkedBlockingQueue is thread-safe by
    // construction, so concurrent presses across floors never corrupt each other.
    private final BlockingQueue<ElevatorRequest> requestQueue = new LinkedBlockingQueue<>();

    ElevatorController(List<ElevatorCar> cars, SchedulingStrategy scheduler) {
        this.cars = cars; this.scheduler = scheduler;
    }

    // Called from any button-press thread. Never blocks the caller on dispatch logic.
    public void submit(ElevatorRequest request) { requestQueue.offer(request); }

    // A single dispatcher thread drains the queue and runs each Command in turn.
    // Serializing execution here – not just queueing submission – is what prevents
    // the real race: two external requests arriving together must not both see the
    // same car as "nearest" and both get assigned it while a third car sits idle.
    public void runDispatchLoop() {
        while (true) {
            try {
                ElevatorRequest request = requestQueue.take(); // blocks until work arrives
                request.execute();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }

    // Assign an external hallway call to the best car (Strategy pattern), then hand
    // it off as an ordinary stop. synchronized so two dispatch() calls in flight
    // can't both read a stale "current position" and pick the same car.
    public synchronized void dispatch(int floor, Direction direction) {
        for (ElevatorCar car : cars) {
            if (floor < car.getMinFloor() || floor > car.getMaxFloor()) {
                throw new IllegalArgumentException("Floor " + floor + " not served by this ba
            }
        }
        ElevatorCar best = scheduler.selectCar(cars, floor, direction);
        if (best == null) {
            throw new IllegalStateException("No elevator available to serve floor " + floor);
        }
        best.addStop(floor);
    }

    public ElevatorCar getCar(int id) {
        return cars.stream().filter(c -> c.getId() == id).findFirst()
            .orElseThrow(() -> new NoSuchElementException("No such car: " + id));
    }
}
```

Why queue and synchronized? The queue absorbs concurrent *submission* (many buttons firing at once) without blocking callers. The single dispatcher thread draining it, plus `synchronized` on

`dispatch`, guarantees requests are *assigned* one at a time — so "find the best car" always sees a consistent snapshot of every car's position before committing to it.

Use-case walkthrough

1. Person on floor 3 presses the hallway UP button.
→ `controller.submit(new ExternalRequest(controller, 3, Direction.UP))`
2. Dispatch loop takes the request, calls `execute()` → `controller.dispatch(3, UP)`
→ `NearestCarStrategy` scans all cars: Car A is idle on floor 1 (cost 2),
Car B is moving UP and already past floor 3 (cost 1000+) → Car A wins.
→ `Car A.addStop(3)`; Car A was Idle, so `IdleState.handle()` fires it into `MovingUpState`.
3. Car A steps: `MovingUpState` pops floor 3, moves there, becomes `DoorsOpenState`.
→ `DoorsOpenState` opens the door; no more stops → Car A goes back to `IdleState`.
4. Person boards, presses floor 7 inside the car.
→ `controller.submit(new InternalRequest(carA, 7))` → `execute()` → `carA.addStop(7)`
→ Idle again, so this wakes Car A straight into `MovingUpState`.
5. Car A steps repeatedly, arrives at 7, `DoorsOpenState` opens the door, then goes Idle.
6. Someone requests floor 99 on a bank whose `maxFloor` is 50
→ `addStop()/dispatch()` throws `IllegalArgumentException` → caller shows an error,
no stop is ever added to a car's queue.

Extensions an interviewer may probe

- **Capacity/weight limits** → `ElevatorCar` tracks current load; `NearestCarStrategy` skips a full car even if it's otherwise the best fit.
 - **Multiple banks serving different floor ranges (low-rise/high-rise)** → controller becomes per-bank; a top-level router picks the bank before picking the car.
 - **Fairness/starvation** → a request that keeps losing to "on the way" cars could wait forever; add an aging factor to `costOf` so wait time lowers cost over time.
-

3. Library Management System

The problem in one line. Design a library system that lets members search the catalog, borrow and return physical copies of books, reserve copies that are checked out, and pay fines for overdue returns.

Step 1 — Clarify requirements

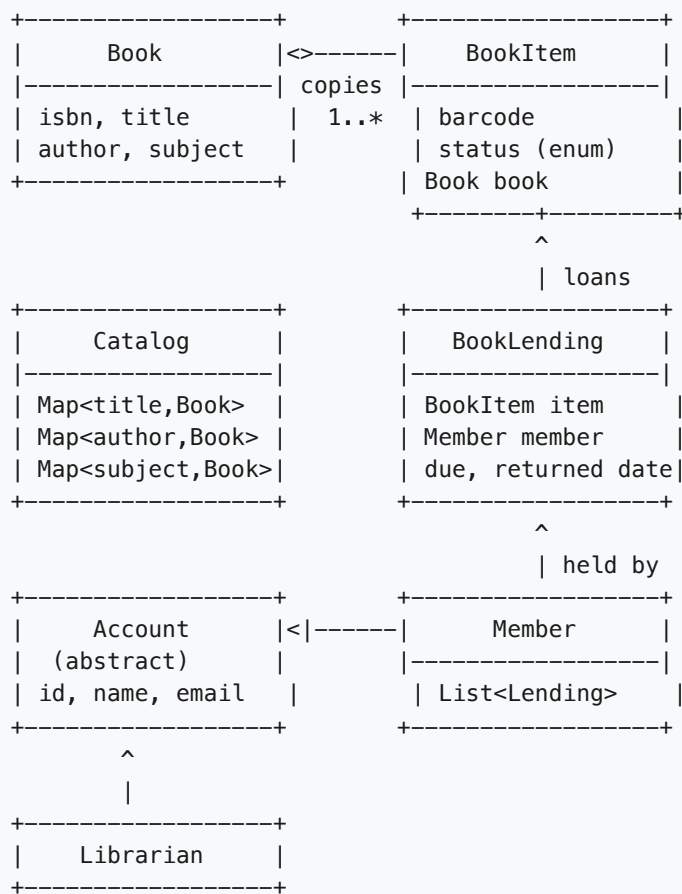
Questions worth asking out loud (and the assumptions this design makes):

- **Book vs. copy?** A `Book` is a title (ISBN, author, subject); a library may own several physical `BookItem`s of the same `Book`, each with its own barcode and condition. *This distinction is the crux of the design — confirm it early.*
- **Search?** By title, author, or subject. Assume reads vastly outnumber writes (new titles/copies), so we optimize for fast lookup over cheap indexing.
- **Borrowing rules?** A member may hold at most `N` books at once (assume 5); each loan has a due date (assume 14 days); returning late accrues a fine.
- **Reservations?** If every copy of a title is checked out, a member can reserve the next one returned — assume simple FIFO, one active hold per member per title.
- **Fines?** Charged per day overdue. Pluggable so the rate/policy can change without touching the lending logic.
- **Concurrency?** Two members may try to borrow the last copy of a title at the same instant — exactly one must succeed.

Step 2 — Entities (the nouns)

`Book`, `BookItem`, `Member`, `Librarian` (+ `Account` base), `Catalog`, `BookLending` (a loan), `Reservation`, `Fine`, `FineStrategy`.

Step 3 — Relationships (class diagram)



FineStrategy (interface) <|-- PerDayFineStrategy [Strategy pattern]

Step 4 — Patterns chosen (and why)

- **Book vs. BookItem** — the single most important modeling call. **Book** holds title-level metadata shared by every copy; **BookItem** holds per-copy state (barcode, condition, availability). Lending, reserving, and locking all happen on **BookItem**, never on **Book** — you don't "borrow a title," you borrow a *copy*.
- **Strategy** for **FineStrategy** — the fine formula (flat per-day, capped, tiered by member type) changes independently of the lending workflow. Open/Closed.
- **Composition** for the catalog: **Catalog** has index maps; **Book** has many **BookItem**s. **Inheritance** only where it earns its keep: **Member** / **Librarian** extend **Account** for shared identity fields.
- **Repository-style indexing** in **Catalog** — three **Map**s (by title, author, subject) trade memory for $O(1)$ average lookup, since search is the hot path and new books are written far less often than they're searched.

Step 5 — Skeleton + key methods

Book and BookItem — the title/copy split

```
enum ItemStatus { AVAILABLE, LOANED, RESERVED }

class Book {
    private final String isbn;
    private final String title;
    private final String author;
    private final String subject;
    private final List<BookItem> copies = new ArrayList<>();

    Book(String isbn, String title, String author, String subject) {
        this.isbn = isbn; this.title = title; this.author = author; this.subject = subject;
    }
    void addCopy(BookItem item) { copies.add(item); }
    List<BookItem> getCopies() { return copies; }
    String getTitle() { return title; }
    String getAuthor() { return author; }
    String getSubject() { return subject; }
}

// The unit of shared, mutable state – one physical copy on a shelf.
class BookItem {
    private final String barcode;
    private final Book book;
    private ItemStatus status = ItemStatus.AVAILABLE;

    BookItem(String barcode, Book book) { this.barcode = barcode; this.book = book; }

    synchronized boolean isAvailable() { return status == ItemStatus.AVAILABLE; }
    synchronized void markLoaned()    { status = ItemStatus.LOANED; }
    synchronized void markAvailable() { status = ItemStatus.AVAILABLE; }
    synchronized void markReserved()  { status = ItemStatus.RESERVED; }

    String getBarcode() { return barcode; }
    Book getBook() { return book; }
}
```

Catalog — indexed search

```
class Catalog {
    // Three indexes traded for memory: search is the hot path, writes are rare.
    private final Map<String, List<Book>> byTitle = new HashMap<>();
    private final Map<String, List<Book>> byAuthor = new HashMap<>();
    private final Map<String, List<Book>> bySubject = new HashMap<>();

    void addBook(Book book) {
        byTitle.computeIfAbsent(book.getTitle().toLowerCase(), k -> new ArrayList<>()).add(book);
        byAuthor.computeIfAbsent(book.getAuthor().toLowerCase(), k -> new ArrayList<>()).add(book);
        bySubject.computeIfAbsent(book.getSubject().toLowerCase(), k -> new ArrayList<>()).add(book);
    }

    List<Book> searchByTitle(String title) { return byTitle.getOrDefault(title.toLowerCase(), new ArrayList<>()); }
    List<Book> searchByAuthor(String author) { return byAuthor.getOrDefault(author.toLowerCase(), new ArrayList<>()); }
    List<Book> searchBySubject(String subj) { return bySubject.getOrDefault(subj.toLowerCase(), new ArrayList<>()); }
}
```

Fine strategy (Strategy pattern)

```
interface FineStrategy {
    double calculate(long daysOverdue);
}

class PerDayFineStrategy implements FineStrategy {
    private final double ratePerDay;
    PerDayFineStrategy(double ratePerDay) { this.ratePerDay = ratePerDay; }

    public double calculate(long daysOverdue) {
        return Math.max(0, daysOverdue) * ratePerDay;    // no fine if returned on/before due
    }
}
```

Accounts, loans, and reservations

```
abstract class Account {
    private final String id;
    private final String name;
    Account(String id, String name) { this.id = id; this.name = name; }
    String getId() { return id; }
}

class BookLending {
    private final BookItem item;
    private final Member member;
    private final LocalDate dueDate;
    private LocalDate returnedDate; // null while outstanding

    BookLending(BookItem item, Member member, LocalDate dueDate) {
        this.item = item; this.member = member; this.dueDate = dueDate;
    }
    BookItem getItem() { return item; }
    LocalDate getDueDate() { return dueDate; }
    void markReturned(LocalDate date) { this.returnedDate = date; }
}

class Reservation {
    final BookItem item;
    final Member member;
    Reservation(BookItem item, Member member) { this.item = item; this.member = member; }
}

class Member extends Account {
    private static final int MAX_BOOKS = 5;
    private final List<BookLending> activeLoans = new ArrayList<>();

    Member(String id, String name) { super(id, name); }
    List<BookLending> getActiveLoans() { return activeLoans; }
    boolean hasReachedLimit() { return activeLoans.size() >= MAX_BOOKS; }
}

class Librarian extends Account {
    Librarian(String id, String name) { super(id, name); }
}
```

The controller — with concurrency + error handling

```
class Library {
    private final Catalog catalog = new Catalog();
    private final Map<String, BookLending> activeLoansByBarcode = new ConcurrentHashMap<>();
    private final Map<String, Deque<Reservation>> reservationsByIsbn = new ConcurrentHashMap<>();
    private FineStrategy fineStrategy = new PerDayFineStrategy(1.0);
    private static final int LOAN_DAYS = 14;

    void setFineStrategy(FineStrategy s) { this.fineStrategy = s; } // swap policy at runtime

    // Borrow: pick an available copy of the book and lend it out.
    // synchronized on the specific BookItem so two members racing for the
    // last copy can't both succeed -- one wins, the other gets a clear error.
    public BookLending borrow(Member member, Book book, LocalDate today) {
        if (member.hasReachedLimit()) {
            throw new IllegalStateException("Member " + member.getId() + " is at the borrow limit");
        }
        BookItem chosen = null;
        for (BookItem candidate : book.getCopies()) {
            synchronized (candidate) {
                if (candidate.isAvailable()) {
                    candidate.markLoaned(); // claim it inside the lock -- no other thread
                    chosen = candidate;
                    break;
                }
            }
        }
        if (chosen == null) {
            throw new NoSuchElementException("No available copy of \"" + book.getTitle() + "\"");
        }
        BookLending lending = new BookLending(chosen, member, today.plusDays(LOAN_DAYS));
        member.getActiveLoans().add(lending);
        activeLoansByBarcode.put(chosen.getBarcode(), lending);
        return lending;
    }

    // Return: validate the item was actually on loan, free it, charge any fine,
    // then hand the copy to the next reservation if one exists.
    public double returnItem(String barcode, LocalDate today) {
        BookLending lending = activeLoansByBarcode.remove(barcode);
        if (lending == null) {
            throw new IllegalStateException("Item " + barcode + " was not on loan");
        }
        lending.markReturned(today);
        BookItem item = lending.getItem();

        double fine = 0.0;
        long daysLate = ChronoUnit.DAYS.between(lending.getDueDate(), today);
        if (daysLate > 0) fine = fineStrategy.calculate(daysLate);

        synchronized (item) {
            Reservation next = pollReservationFor(item);
            if (next != null) {
                item.markReserved(); // hand straight to the next member in line
            } else {
                item.markAvailable();
            }
        }
        return fine;
    }
}
```



```

    }

    // Reserve: only allowed when every copy of the book is currently loaned out.
    public Reservation reserve(Member member, Book book) {
        boolean anyAvailable = book.getCopies().stream().anyMatch(BookItem::isAvailable);
        if (anyAvailable) {
            throw new IllegalStateException("Copies are available -- borrow instead of reserv
        }
        BookItem target = book.getCopies().get(0);    // FIFO hold against the title's queue
        Reservation reservation = new Reservation(target, member);
        reservationsByIsbn
            .computeIfAbsent(book.getCopies().get(0).getBook().getTitle(), k -> new ArrayDeque<Reservation>())
            .addLast(reservation);
        return reservation;
    }

    private Reservation pollReservationFor(BookItem item) {
        Deque<Reservation> queue = reservationsByIsbn.get(item.getBook().getTitle());
        return (queue == null || queue.isEmpty()) ? null : queue.pollFirst();
    }
}

```

Why lock on the `BookItem`, not the whole `Library`? The race is narrow: two threads iterating the same `Book`'s copies could both see the same `BookItem` as `AVAILABLE` and both try to lend it. Locking each candidate copy while checking and claiming it keeps contention scoped to a single copy instead of serializing every borrow in the building — unrelated titles never block each other.

Use-case walkthrough

1. `Book clean = new Book("978-1", "Clean Code", "R. Martin", "Software");`
`catalog.addBook(clean);`
`clean.addCopy(new BookItem("BC-001", clean));` // only one copy in stock
2. `List<Book> hits = catalog.searchByTitle("Clean Code");` // O(1) map lookup
3. `BookLending loan = library.borrow(alice, clean, today);`
 -> scans copies, locks BC-001, it's AVAILABLE -> marks LOANED
 -> due date = today + 14 days; `alice.activeLoans` now has 1 entry
4. `library.borrow(bob, clean, today);` // races alice for the same title
 -> scans copies, BC-001 is already LOANED -> no available copy found
 -> throws `NoSuchElementException("No available copy of \"Clean Code\"")`
5. `Reservation r = library.reserve(bob, clean);`
 -> all copies loaned -> bob is queued FIFO against "Clean Code"
6. `double fine = library.returnItem("BC-001", today.plusDays(20));`
 -> `daysLate = 20 - 14 = 6` -> `PerDayFineStrategy: 6 * 1.0 = 6.0`
 -> `pollReservationFor` finds bob's hold -> item marked RESERVED for bob

Extensions an interviewer may probe

- **Notify members when a reserved copy is ready** → add an `Observer` so waiting members get an email/SMS the moment `pollReservationFor` finds their hold.

- **Different loan periods by member tier** (student vs. faculty) → pull `LOAN_DAYS` into a per-`Account` policy, another `Strategy` .
 - **Multiple branches sharing a catalog** → `Catalog` becomes a facade over a per-branch inventory; `BookItem` gains a `branchId` , and reservations must decide whether holds are branch-scoped or system-wide.
-

4. Vending Machine

The problem in one line. Design a vending machine that accepts coins, lets a user select a product, dispenses it and returns change — while behaving correctly regardless of what order the buttons are pressed.

Step 1 — Clarify requirements

Questions worth asking out loud (and the assumptions this design makes):

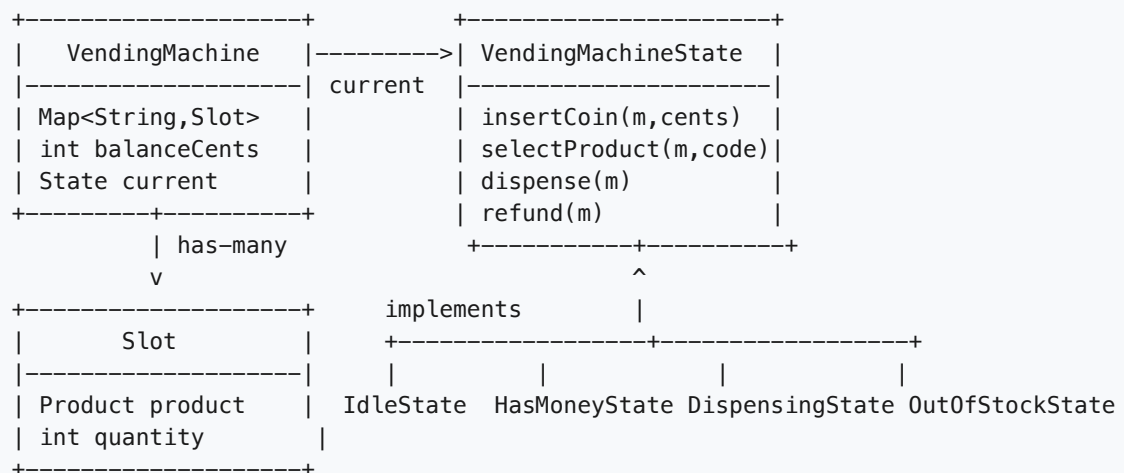
- **Payment method?** Coins only, tracked as cents (bills/cards are an extension).
- **Inventory?** Machine has fixed slots, each holding one `Product` type and a count.
- **Illegal actions?** Selecting a product before inserting money, or inserting money after a selection is already being dispensed, must be rejected cleanly — not crash the machine.
- **Out of stock?** Selecting a sold-out slot must refuse and say so, not accept money for nothing.
- **Change?** If inserted money exceeds price, return the difference; if it's insufficient, either ask for more or let the user cancel and get a refund.
- **Concurrency?** A real machine has one keypad/coin slot, but the controller must not let two near-simultaneous selections both decrement the same last unit of stock.

The behavior above — *what a button press does depends entirely on where the machine currently is in its lifecycle* — is the textbook signal for the **State pattern**, which is the centerpiece of this design.

Step 2 — Entities (the nouns)

`VendingMachine`, `VendingMachineState` (+ concrete states), `Product`, `Slot` (inventory row), `Coin`.

Step 3 — Relationships (class diagram)



[State pattern] each concrete state decides what `insertCoin/select/dispense/refund` actually do while the machine is "in" that state.

Step 4 — Patterns chosen (and why)

- **State** for the machine's lifecycle (`Idle` → `HasMoney` → `Dispensing` → back to `Idle`, with `OutOfStock` as a guard state) — this is the problem. Each state class owns the behavior legal in that state; the machine just delegates and holds a reference to "current state." No giant `switch` on an enum scattered through every method.
- **Singleton**-ish controller — one `VendingMachine` instance coordinates all state and inventory; not a hard requirement but common in these designs.
- **Strategy** (implicit) for change-making — `makeChange` is isolated so denomination logic can change without touching state classes.
- **Encapsulation** — `Slot` hides quantity bookkeeping; states never touch inventory maps directly, always through the machine's methods.

Step 5 — Skeleton + key methods

Product and Slot

```
class Product {
    private final String name;
    private final int priceCents;
    Product(String name, int priceCents) { this.name = name; this.priceCents = priceCents; }
    public int getPriceCents() { return priceCents; }
    public String getName() { return name; }
}

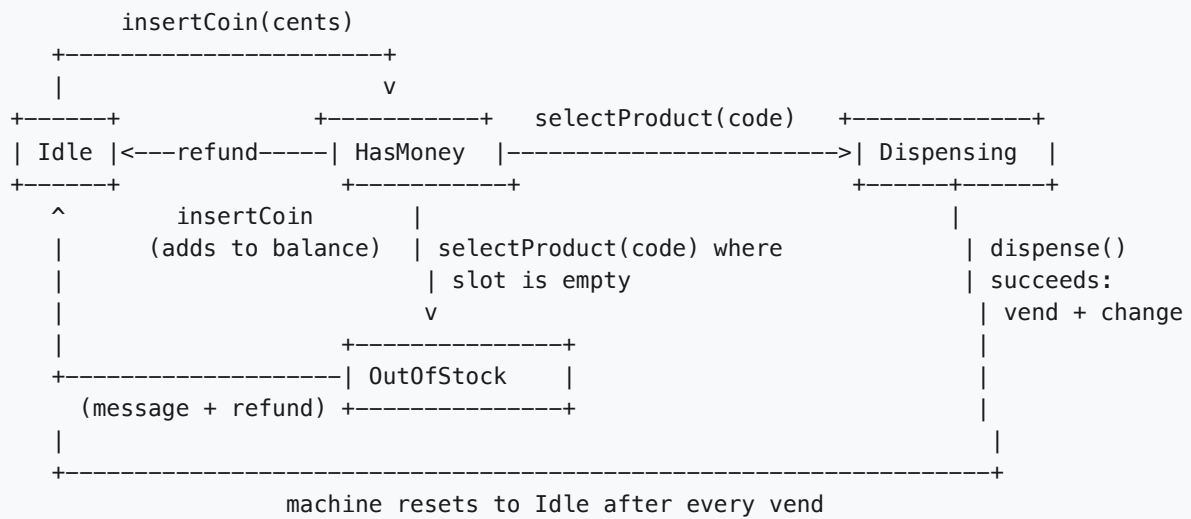
class Slot {
    private final Product product;
    private int quantity;
    Slot(Product product, int quantity) { this.product = product; this.quantity = quantity; }

    boolean isEmpty()      { return quantity <= 0; }
    Product getProduct()   { return product; }
    void decrement()       { quantity--; }
}
```

State interface — the centerpiece

```
// Every state answers the same four questions differently.
// Default methods reject the action; concrete states override what's legal.
interface VendingMachineState {
    default void insertCoin(VendingMachine m, int cents) {
        throw new IllegalStateException("Cannot insert coins now: " + m.stateName());
    }
    default void selectProduct(VendingMachine m, String code) {
        throw new IllegalStateException("Cannot select a product now: " + m.stateName());
    }
    default void dispense(VendingMachine m) {
        throw new IllegalStateException("Cannot dispense now: " + m.stateName());
    }
    default void refund(VendingMachine m) {
        throw new IllegalStateException("Nothing to refund: " + m.stateName());
    }
}
```

State transition diagram



Concrete states

```
class IdleState implements VendingMachineState {
    @Override public void insertCoin(VendingMachine m, int cents) {
        m.addBalance(cents);                // start a transaction
        m.setState(m.getHasMoneyState());
    }
    // selectProduct/dispense/refund fall through to the interface's "illegal" defaults –
    // you can't buy anything before paying.
}

class HasMoneyState implements VendingMachineState {
    @Override public void insertCoin(VendingMachine m, int cents) {
        m.addBalance(cents);                // top up while deciding
    }

    @Override public void selectProduct(VendingMachine m, String code) {
        Slot slot = m.getSlot(code);        // throws if code is invalid
        if (slot.isEmpty()) {
            m.setState(m.getOutOfStockState());
            throw new NoSuchElementException("Out of stock: " + code);
        }
        if (m.getBalanceCents() < slot.getProduct().getPriceCents()) {
            int missing = slot.getProduct().getPriceCents() - m.getBalanceCents();
            throw new IllegalArgumentException("Insufficient funds, insert " + missing + " mo
        }
        m.setSelectedCode(code);
        m.setState(m.getDispensingState());
        m.dispense();                        // auto-advance to vending
    }

    @Override public void refund(VendingMachine m) {
        m.returnChange(m.getBalanceCents()); // give back everything inserted so far
        m.resetTransaction();
        m.setState(m.getIdleState());
    }
}

class DispensingState implements VendingMachineState {
    @Override public void dispense(VendingMachine m) {
        Slot slot = m.getSlot(m.getSelectedCode());
        int price = slot.getProduct().getPriceCents();
        int change = m.getBalanceCents() - price;

        slot.decrement();                    // vend
        m.returnChange(change);               // may be zero
        m.resetTransaction();
        m.setState(m.getIdleState());        // ready for the next customer
    }
    // insertCoin/selectProduct/refund are illegal mid-dispense: the interface defaults apply
}

class OutOfStockState implements VendingMachineState {
    @Override public void refund(VendingMachine m) {
        m.returnChange(m.getBalanceCents());
        m.resetTransaction();
        m.setState(m.getIdleState());
    }
    // insertCoin/selectProduct stay illegal until refund clears the transaction –
```

```
} // forces the user to take their money back before trying again.
```

VendingMachine — context + inventory + concurrency

```
class VendingMachine {
    private final Map<String, Slot> slots = new HashMap<>();
    private int balanceCents = 0;
    private String selectedCode;

    // One instance per state – states are stateless, so they're safe to share/reuse.
    private final VendingMachineState idleState = new IdleState();
    private final VendingMachineState hasMoneyState = new HasMoneyState();
    private final VendingMachineState dispensingState = new DispensingState();
    private final VendingMachineState outOfStockState = new OutOfStockState();
    private VendingMachineState current = idleState;

    public void loadSlot(String code, Product product, int qty) {
        slots.put(code, new Slot(product, qty));
    }

    // synchronized: two customers hitting insertCoin/select at once must not
    // interleave balance updates or both dispense the same last item.
    public synchronized void insertCoin(int cents) { current.insertCoin(this, cents); }
    public synchronized void selectProduct(String code) { current.selectProduct(this, code); }
    public synchronized void dispense() { current.dispense(this); }
    public synchronized void refund() { current.refund(this); }

    Slot getSlot(String code) {
        Slot slot = slots.get(code);
        if (slot == null) throw new NoSuchElementException("Invalid selection: " + code);
        return slot;
    }

    void addBalance(int cents) { balanceCents += cents; }
    int getBalanceCents() { return balanceCents; }
    void setSelectedCode(String code) { this.selectedCode = code; }
    String getSelectedCode() { return selectedCode; }
    void resetTransaction() { balanceCents = 0; selectedCode = null; }

    void returnChange(int cents) {
        if (cents > 0) System.out.println("Returning change: " + cents + " cents");
    }

    void setState(VendingMachineState s) { this.current = s; }
    VendingMachineState getIdleState() { return idleState; }
    VendingMachineState getHasMoneyState() { return hasMoneyState; }
    VendingMachineState getDispensingState() { return dispensingState; }
    VendingMachineState getOutOfStockState() { return outOfStockState; }
    String stateName() { return current.getClass().getSimpleName(); }
}
```

Why `synchronized` on the four public entry points? Coin insertion, selection and dispensing all read-then-write shared mutable state (`balanceCents` , `slots` , `current`). Without a lock, two threads could both pass the "slot not empty" check on the last unit and both decrement it, or one thread's `refund` could race another's `insertCoin` and corrupt the balance. Locking the whole machine is the simple,

correct answer for a single physical machine with one coin slot and one keypad — there's no meaningful concurrency to preserve inside one machine anyway.

Use-case walkthrough

```
1. VendingMachine m = new VendingMachine();
   m.loadSlot("A1", new Product("Soda", 125), 3);
2. m.insertCoin(100); // IdleState.insertCoin -> balance=100, state=HasMoney
3. m.insertCoin(50);  // HasMoneyState.insertCoin -> balance=150
4. m.selectProduct("A1");
   -> HasMoneyState.selectProduct: slot not empty, balance(150) >= price(125)
   -> state=Dispensing, auto-calls dispense()
   -> DispensingState.dispense: slot.decrement() (3->2), change=25 returned
   -> resetTransaction(); state=Idle
5. Next customer selects "A1" with balance=0
   -> would be in IdleState -> selectProduct() hits interface default
   -> throws IllegalStateException("Cannot select a product now: IdleState")
6. Slot eventually empties (qty=0); a later selectProduct() call
   -> HasMoneyState.selectProduct: slot.isEmpty() -> state=OutOfStock, throws
   -> caller must call refund() before the machine accepts coins again.
```

Extensions an interviewer may probe

- **Multiple payment methods (card, mobile pay)** → introduce a `PaymentStrategy` alongside the state machine; states call `strategy.charge()` instead of touching coins directly.
 - **Restocking / admin mode** → add an `AdminState` reachable only via a key, exposing `refill(code, qty)` while blocking customer actions.
 - **Exact-change-only mode** → before entering `Dispensing`, check whether the machine's coin reserve can make the required change; if not, reject the sale even with enough funds.
-

5. Movie Ticket Booking (BookMyShow)

The problem in one line. Design a system where users browse shows across cities and cinemas, pick seats for a specific show, and pay — while guaranteeing that two users can never end up holding or paying for the same seat.

Step 1 — Clarify requirements

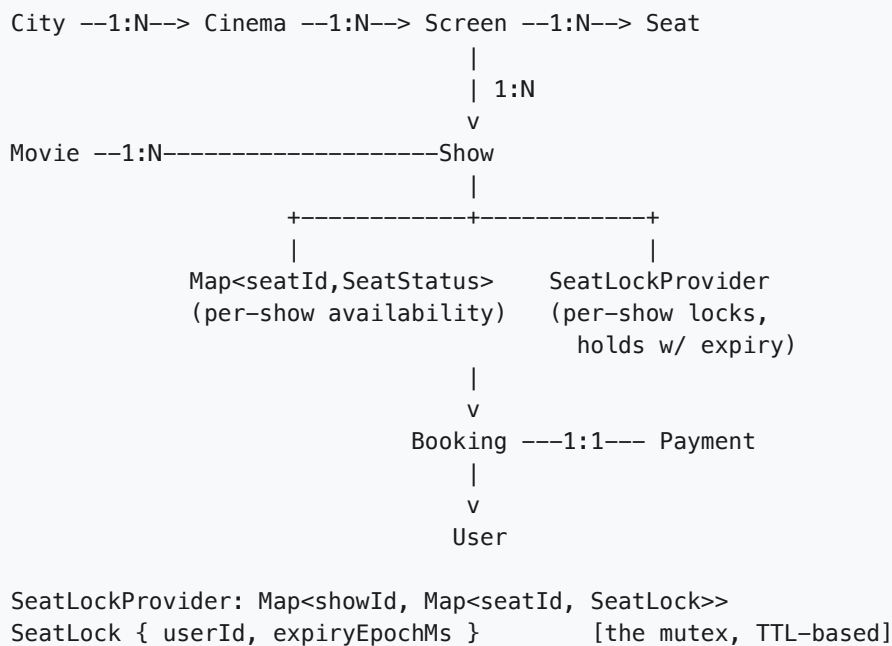
Questions worth asking out loud (and the assumptions this design makes):

- **Scope of browsing?** City → Cinema → Movie → Show. A `Show` is one screening of one `Movie` in one `Screen` at one time.
- **Seat layout?** Each `Screen` has a fixed grid of `Seat`s, each with a `SeatType` (REGULAR, PREMIUM, RECLINER) that drives price. *Seat availability* is per- `Show` , not per- `Screen` (the same physical seat is free for the 3 PM show and booked for the 6 PM show).
- **Booking flow?** User selects seats → seats are held for them for a short window (say 5 minutes) → user pays → booking confirms. If payment doesn't happen in time, the hold expires and seats return to the pool.
- **Concurrency — the core of this problem.** Two users clicking the same seat for the same show at the same instant must not both succeed. Exactly one gets the seat; the other is told it's taken, ideally *before* they waste time on the payment page.
- **Payment?** Out of scope for correctness of the payment gateway itself — assume a `PaymentGateway` that returns success/failure; we own the state machine around it.
- **Overselling under load?** Must be impossible even with many concurrent booking threads for the same show — this is the property the design is graded on.

Step 2 — Entities (the nouns)

`City` , `Cinema` , `Screen` , `Movie` , `Show` , `Seat` , `SeatType` , `User` , `Booking` , `Payment` , `SeatLockProvider` .

Step 3 — Relationships (class diagram)



Step 4 — Patterns chosen (and why)

- **State** for `Booking` — `CREATED` → `SEATS_LOCKED` → `PAYMENT_PENDING` → `CONFIRMED` / `EXPIRED` is a strict state machine; illegal transitions (e.g. paying for an `EXPIRED` booking) throw instead of silently succeeding.
- **Strategy-free, but pluggable** `PaymentGateway` — an interface so the gateway implementation (Stripe/Razorpay/mock) can be swapped without touching booking logic.
- **Observer** for post-confirmation notification — `Booking` notifies subscribers (email/SMS service) on `CONFIRMED`; the booking flow doesn't know or care who's listening.
- **Facade** — `BookingService` is the single entry point coordinating `SeatLockProvider`, `Show`, `PaymentGateway`, and `Booking`, so callers never juggle those objects directly.
- **Pessimistic locking (not optimistic) for seats** — chosen deliberately; see Step 5.

Step 5 — Skeleton + key methods

Core value types

```
enum SeatType { REGULAR, PREMIUM, RECLINER }

enum SeatStatus { AVAILABLE, LOCKED, BOOKED }

enum BookingStatus { CREATED, SEATS_LOCKED, PAYMENT_PENDING, CONFIRMED, EXPIRED, CANCELLED }

class Seat {
    private final String id;           // e.g. "A1"
    private final SeatType type;
    Seat(String id, SeatType type) { this.id = id; this.type = type; }
    String getId() { return id; }
    SeatType getType() { return type; }
}

class Movie {
    private final String id;
    private final String title;
    private final int durationMins;
    Movie(String id, String title, int durationMins) {
        this.id = id; this.title = title; this.durationMins = durationMins;
    }
}

class Screen {
    private final String id;
    private final List<Seat> seats;
    Screen(String id, List<Seat> seats) { this.id = id; this.seats = seats; }
    List<Seat> getSeats() { return seats; }
}

class Cinema {
    private final String id;
    private final String name;
    private final List<Screen> screens;
    Cinema(String id, String name, List<Screen> screens) {
        this.id = id; this.name = name; this.screens = screens;
    }
}
```

Show — the unit that owns per-screening seat state

```
class Show {
    private final String id;
    private final Movie movie;
    private final Screen screen;
    private final long startEpochMs;
    private final Map<SeatType, Double> priceByType;
    // Per-show availability; this map is the thing concurrent bookings fight over.
    private final Map<String, SeatStatus> seatStatus = new ConcurrentHashMap<>();

    Show(String id, Movie movie, Screen screen, long startEpochMs,
        Map<SeatType, Double> priceByType) {
        this.id = id; this.movie = movie; this.screen = screen;
        this.startEpochMs = startEpochMs; this.priceByType = priceByType;
        for (Seat s : screen.getSeats()) seatStatus.put(s.getId(), SeatStatus.AVAILABLE);
    }

    String getId() { return id; }
    SeatStatus statusOf(String seatId) { return seatStatus.get(seatId); }
    void markLocked(String seatId) { seatStatus.put(seatId, SeatStatus.LOCKED); }
    void markBooked(String seatId) { seatStatus.put(seatId, SeatStatus.BOOKED); }
    void markAvailable(String seatId) { seatStatus.put(seatId, SeatStatus.AVAILABLE); }
    double priceOf(Seat seat) { return priceByType.get(seat.getType()); }
}
```

The race, precisely

Two threads call `bookSeats(showId, {"A1","A2"}, userA)` and `bookSeats(showId, {"A2","A3"}, userB)` at the same moment. Both threads read `seatStatus.get("A2")`, both see `AVAILABLE`, both proceed to lock it — now seat A2 is "held" by two users and whoever pays first still leaves the other with a confirmed-looking hold that later fails. The critical section is **check-then-set on the shared per-show seat map**, and it must be atomic per seat, across all threads, for the lifetime of the hold (not just the single map write).

Pessimistic vs optimistic: optimistic locking (version-stamp the seat row, fail-and-retry on conflict) works for a single write, but here the "lock" must *survive* for the whole seat-selection → payment window (minutes), not just one transaction. A version check can't express "hold this for 5 minutes unless someone pays." So this design uses **pessimistic, time-boxed locks**: acquire a real lock per seat before letting the user proceed to payment, and release it automatically via expiry if they never pay. That also fails fast — the second user is told "seat taken" at *selection* time, not after filling in card details.

SeatLockProvider — the mutex with a timeout

```
class SeatLock {
    final String userId;
    final long expiryEpochMs;
    SeatLock(String userId, long expiryEpochMs) {
        this.userId = userId; this.expiryEpochMs = expiryEpochMs;
    }
    boolean isExpired(long nowMs) { return nowMs >= expiryEpochMs; }
}

// One lock table per show; ReentrantLock guards the compound
// check-expiry-then-insert operation on each seat's entry.
class SeatLockProvider {
    private static final long HOLD_MS = 5 * 60 * 1000L;    // 5-minute hold

    // showId -> seatId -> current lock (absent = free)
    private final Map<String, Map<String, SeatLock>> locksByShow = new ConcurrentHashMap<>();
    // one physical lock per (show,seat) to serialize the critical section below
    private final Map<String, ReentrantLock> mutexes = new ConcurrentHashMap<>();

    // Try to lock every requested seat for this user. All-or-nothing:
    // if any seat is taken, roll back the ones we already grabbed.
    public void lockSeats(String showId, List<String> seatIds, String userId) {
        Map<String, SeatLock> showLocks =
            locksByShow.computeIfAbsent(showId, k -> new ConcurrentHashMap<>());
        List<String> acquired = new ArrayList<>();
        long now = System.currentTimeMillis();

        try {
            for (String seatId : seatIds) {
                ReentrantLock mutex = mutexes.computeIfAbsent(
                    showId + ":" + seatId, k -> new ReentrantLock());
                mutex.lock();                                // <-- the critical section starts here
                try {
                    SeatLock existing = showLocks.get(seatId);
                    boolean free = existing == null || existing.isExpired(now);
                    if (!free) {
                        throw new SeatAlreadyLockedException(seatId);
                    }
                    // check-and-set is atomic because we hold the per-seat mutex
                    showLocks.put(seatId, new SeatLock(userId, now + HOLD_MS));
                    acquired.add(seatId);
                } finally {
                    mutex.unlock();                            // <-- critical section ends here
                }
            }
        } catch (SeatAlreadyLockedException ex) {
            // partial failure: release whatever this call already grabbed
            for (String seatId : acquired) showLocks.remove(seatId);
            throw ex;
        }
    }

    // Called on payment success: converts a hold into a permanent booking.
    // Fails loudly if the hold expired mid-payment (someone else may now own it).
    public void confirmAndRelease(String showId, String seatId, String userId) {
        Map<String, SeatLock> showLocks = locksByShow.get(showId);
        SeatLock lock = showLocks == null ? null : showLocks.get(seatId);
        long now = System.currentTimeMillis();
    }
}
```

```

        if (lock == null || lock.isExpired(now) || !lock.userId.equals(userId)) {
            throw new SeatLockExpiredException(seatId);
        }
        showLocks.remove(seatId); // seat now transitions to BOOKED, not back to AVAILABLE
    }

    // Called on payment failure/timeout: just drop the hold, seat becomes free again.
    public void release(String showId, String seatId, String userId) {
        Map<String, SeatLock> showLocks = locksByShow.get(showId);
        if (showLocks == null) return;
        SeatLock lock = showLocks.get(seatId);
        if (lock != null && lock.userId.equals(userId)) showLocks.remove(seatId);
    }
}

class SeatAlreadyLockedException extends RuntimeException {
    SeatAlreadyLockedException(String seatId) { super("Seat already locked/booked: " + seatId) }
}

class SeatLockExpiredException extends RuntimeException {
    SeatLockExpiredException(String seatId) { super("Lock expired before payment: " + seatId) }
}

```

Why a ReentrantLock per (show, seat) instead of one big lock? Locking the whole show (or worse, the whole system) would serialize bookings for unrelated seats and kill throughput on a popular show. Locking per seat means users booking A1/A2 and users booking C5/C6 never block each other — only genuine contention on the *same* seat blocks. The `ConcurrentHashMap.computeIfAbsent` for both `locksByShow` and `mutexes` is itself thread-safe, so creating a show's lock table or a seat's mutex on first use is race-free.

Why not synchronized? `ReentrantLock` lets us bound wait time (`tryLock(timeout)`) if we later want to fail fast under contention instead of blocking indefinitely — a reasonable upgrade an interviewer may ask for.

Booking — the state machine

```
class Booking {
    private final String id;
    private final String userId;
    private final Show show;
    private final List<String> seatIds;
    private BookingStatus status;
    private final List<BookingObserver> observers = new ArrayList<>();

    Booking(String id, String userId, Show show, List<String> seatIds) {
        this.id = id; this.userId = userId; this.show = show; this.seatIds = seatIds;
        this.status = BookingStatus.CREATED;
    }

    void addObserver(BookingObserver o) { observers.add(o); }

    // Guarded transition: only valid from the state the caller expects.
    void transition(BookingStatus from, BookingStatus to) {
        if (status != from) {
            throw new IllegalStateException("Expected " + from + " but was " + status);
        }
        status = to;
        if (to == BookingStatus.CONFIRMED) {
            for (BookingObserver o : observers) o.onConfirmed(this);    // Observer pattern
        }
    }

    BookingStatus getStatus() { return status; }
    List<String> getSeatIds() { return seatIds; }
    Show getShow() { return show; }
}

interface BookingObserver { void onConfirmed(Booking booking); }

class NotifyUserObserver implements BookingObserver {
    public void onConfirmed(Booking booking) {
        // send email/SMS/push — decoupled from booking logic
        System.out.println("Booking " + booking.getShow().getId() + " confirmed.");
    }
}
```


BookingService — the Facade tying it together

```
interface PaymentGateway { boolean charge(String userId, double amount); }

class BookingService {
    private final SeatLockProvider lockProvider = new SeatLockProvider();
    private final PaymentGateway paymentGateway;
    private final Map<String, Booking> bookings = new ConcurrentHashMap<>();
    private final AtomicLong bookingSeq = new AtomicLong(0);

    BookingService(PaymentGateway gateway) { this.paymentGateway = gateway; }

    // Step 1: hold the seats. Throws SeatAlreadyLockedException on conflict.
    public Booking holdSeats(Show show, List<String> seatIds, String userId) {
        lockProvider.lockSeats(show.getId(), seatIds, userId);
        for (String seatId : seatIds) show.markLocked(seatId);

        Booking booking = new Booking(
            "BKG" + bookingSeq.incrementAndGet(), userId, show, seatIds);
        booking.addObserver(new NotifyUserObserver());
        booking.transition(BookingStatus.CREATED, BookingStatus.SEATS_LOCKED);
        bookings.put(booking.getId(), booking);
        return booking;
    }

    // Step 2: pay. On success, locks convert to permanent bookings.
    // On lock expiry mid-payment, refund and surface the error – never
    // silently mark the booking confirmed for a seat we no longer own.
    public void pay(String bookingId, double amount) {
        Booking booking = bookings.get(bookingId);
        if (booking == null) throw new NoSuchElementException("Unknown booking: " + bookingId);

        booking.transition(BookingStatus.SEATS_LOCKED, BookingStatus.PAYMENT_PENDING);

        boolean charged = paymentGateway.charge(booking.getUserId(), amount);
        if (!charged) {
            releaseAll(booking);
            booking.transition(BookingStatus.PAYMENT_PENDING, BookingStatus.CANCELLED);
            throw new PaymentFailedException(bookingId);
        }

        try {
            for (String seatId : booking.getSeatIds()) {
                lockProvider.confirmAndRelease(booking.getShow().getId(), seatId, booking.getShow().markBooked(seatId));
            }
        } catch (SeatLockExpiredException ex) {
            // Payment succeeded but our hold lapsed first – must not keep the seat
            // or the user's money silently. Refund and fail the booking.
            paymentGateway.charge(booking.getUserId(), -amount); // refund
            booking.transition(BookingStatus.PAYMENT_PENDING, BookingStatus.EXPIRED);
            throw ex;
        }

        booking.transition(BookingStatus.PAYMENT_PENDING, BookingStatus.CONFIRMED);
    }

    // Background sweeper (scheduled every N seconds) calls this for stale holds.
    public void expireIfStale(String bookingId) {
        Booking booking = bookings.get(bookingId);
```

```

        if (booking == null || booking.getStatus() != BookingStatus.SEATS_LOCKED) return;
        releaseAll(booking);
        booking.transition(BookingStatus.SEATS_LOCKED, BookingStatus.EXPIRED);
    }

    private void releaseAll(Booking booking) {
        for (String seatId : booking.getSeatIds()) {
            lockProvider.release(booking.getShow().getId(), seatId, booking.getUserId());
            booking.getShow().markAvailable(seatId);
        }
    }
}

class PaymentFailedException extends RuntimeException {
    PaymentFailedException(String bookingId) { super("Payment failed for booking: " + bookingId); }
}

```

Error handling summary: `SeatAlreadyLockedException` (seat taken by someone else), `SeatLockExpiredException` (hold lapsed before/during payment — refund and fail rather than double-book), `PaymentFailedException` (gateway declined — release seats immediately so others aren't blocked by a doomed booking), `IllegalStateException` (an out-of-order transition, e.g. paying twice for the same booking).

Use-case walkthrough

1. userA browses City → Cinema → Movie → picks Show S1.
2. userA calls `holdSeats(S1, ["A1","A2"], userA)`
 - lockProvider grabs per-seat mutexes for A1, A2 in order
 - both AVAILABLE → both locked with 5-min expiry → Booking SEATS_LOCKED
3. userB concurrently calls `holdSeats(S1, ["A2","A3"], userB)`
 - mutex for A2 briefly contested; sees an active, non-expired lock
 - throws `SeatAlreadyLockedException("A2")` → rolls back any grabbed seats
 - userB is told "A2 just got taken" immediately, before entering payment
4. userA calls `pay(bookingId, 500.0)` → gateway charges successfully
 - `confirmAndRelease` for A1, A2 → Show marks them BOOKED
 - Booking → CONFIRMED → `NotifyUserObserver` fires
5. userC never pays; 5 minutes pass; scheduler calls `expireIfStale(bookingId)`
 - seats released back to AVAILABLE for the next booker.

Extensions an interviewer may probe

- **Waitlist for sold-out shows** → Observer again: notify waitlisted users when a `CONFIRMED` booking is cancelled and a seat frees up.
- **Dynamic/surge pricing** → swap the flat `priceByType` map for a `PricingStrategy` (Strategy pattern), same shape as the parking lot's `FeeStrategy`.
- **Distributed deployment (multiple app servers)** → the in-memory `SeatLockProvider` must move to a shared store (Redis `SETNX` with TTL, or a DB row with a lease column) since `ConcurrentHashMap` only guards one JVM — the concurrency *design* stays identical, only the lock's storage changes.

6. Splitwise (Expense Sharing)

The problem in one line. Design an expense-sharing system where users in a group can add shared expenses, split them equally/exactly/by percentage, and see who owes whom — with support for simplifying everyone's debts down to the fewest payments.

Step 1 — Clarify requirements

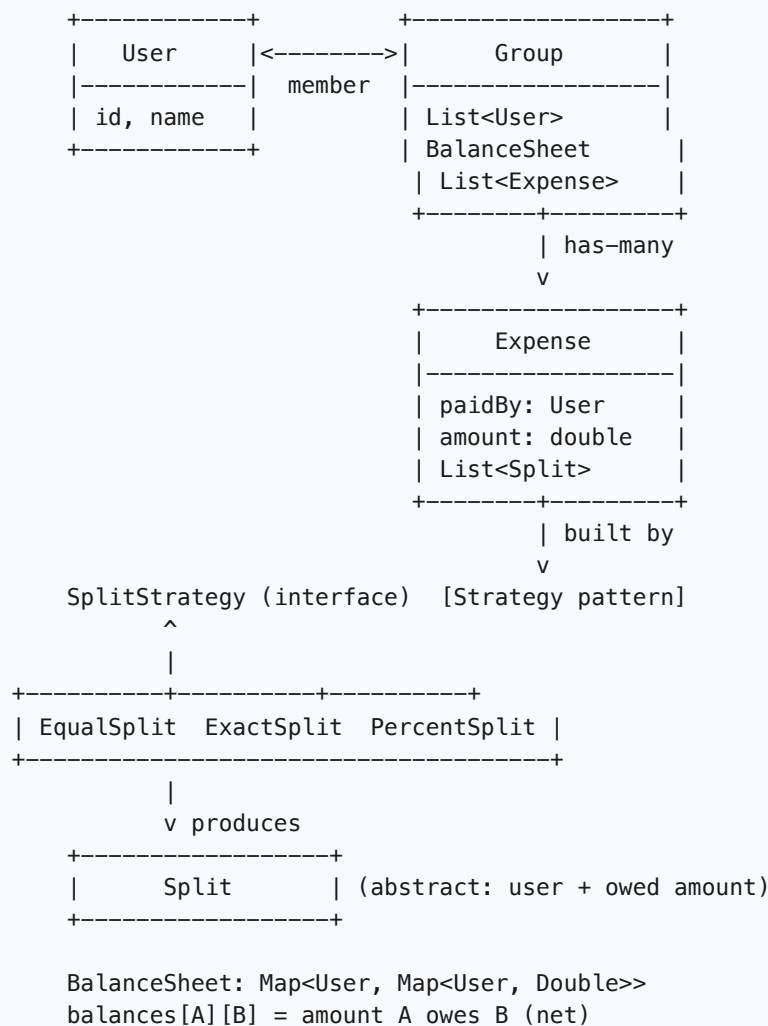
Questions worth asking out loud (and the assumptions this design makes):

- **Split types?** Equal, Exact (explicit amounts), Percent (explicit %). *Pluggable* — *new split types should not touch existing code.*
- **Scope of an expense?** Always inside a `Group`, paid by one `User`, shared among a subset of that group's members (defaults to all members).
- **What does the system report?** Pairwise balances ("A owes B \$30") and, on request, a simplified set of settle-up transactions.
- **Validation?** Exact splits must sum to the total amount; percent splits must sum to 100%; every participant must be a real group member. Bad input → exception, not a silently wrong balance.
- **Concurrency?** Two expenses can be added to the same group at the same instant (e.g. from two phones) — balance updates must not interleave and corrupt the sheet.
- **Settling?** A payment between two users zeroes out (or reduces) their pairwise debt.

Step 2 — Entities (the nouns)

`User`, `Group`, `Expense`, `Split` (abstract, one of `EqualSplit` / `ExactSplit` / `PercentSplit`), `SplitStrategy` (creates the splits), `BalanceSheet`.

Step 3 — Relationships (class diagram)



Step 4 — Patterns chosen (and why)

- **Strategy** for `SplitStrategy` — equal/exact/percent computation is an algorithm that varies independently of `Expense`; a new split type (e.g. "by shares") plugs in without touching `Group` or `Expense`. Open/Closed in action.
- **Factory** (`SplitStrategyFactory`) to pick the right strategy from an enum/type string so callers never `new` a concrete strategy.
- **Composition** — a `Group` *has* `User` s and `Expense` s; an `Expense` *has* `Split` s. No inheritance needed between these.
- **Encapsulation of shared mutable state** — all balance mutation goes through one `BalanceSheet` object per group, which is the single thing we synchronize on.

Step 5 — Skeleton + key methods

User and Split hierarchy

```
class User {
    private final String id;
    private final String name;
    User(String id, String name) { this.id = id; this.name = name; }
    String getId() { return id; }
    public String toString() { return name; }
}

// One user's share of one expense.
abstract class Split {
    protected final User user;
    protected double amount; // resolved amount owed for this split
    Split(User user) { this.user = user; }
    User getUser() { return user; }
    double getAmount() { return amount; }
    void setAmount(double amount) { this.amount = amount; }
}

class EqualSplit extends Split { EqualSplit(User u) { super(u); } }
class ExactSplit extends Split { ExactSplit(User u, double amt) { super(u); this.amount = amt; } }
class PercentSplit extends Split {
    private final double percent;
    PercentSplit(User u, double percent) { super(u); this.percent = percent; }
    double getPercent() { return percent; }
}
```

SplitStrategy (Strategy pattern) — with validation

```
interface SplitStrategy {
    // Resolve each split's final `amount`, validating against the total.
    List<Split> compute(double totalAmount, List<Split> splits);
}

class EqualSplitStrategy implements SplitStrategy {
    public List<Split> compute(double total, List<Split> splits) {
        double share = round2(total / splits.size());
        for (Split s : splits) s.setAmount(share);
        return splits; // last-cent rounding tolerated by the epsilon check below
    }
}

class ExactSplitStrategy implements SplitStrategy {
    public List<Split> compute(double total, List<Split> splits) {
        double sum = splits.stream().mapToDouble(Split::getAmount).sum();
        if (Math.abs(sum - total) > 0.01) {
            throw new IllegalArgumentException(
                "Exact splits (" + sum + ") do not sum to total (" + total + ")");
        }
        return splits;
    }
}

class PercentSplitStrategy implements SplitStrategy {
    public List<Split> compute(double total, List<Split> splits) {
        double pctSum = splits.stream()
            .mapToDouble(s -> ((PercentSplit) s).getPercent()).sum();
        if (Math.abs(pctSum - 100.0) > 0.01) {
            throw new IllegalArgumentException("Percentages sum to " + pctSum + ", not 100");
        }
        for (Split s : splits) {
            double pct = ((PercentSplit) s).getPercent();
            s.setAmount(round2(total * pct / 100.0));
        }
        return splits;
    }
}

// Factory: callers ask for a strategy by type, never construct one directly.
class SplitStrategyFactory {
    static SplitStrategy get(SplitType type) {
        switch (type) {
            case EQUAL: return new EqualSplitStrategy();
            case EXACT: return new ExactSplitStrategy();
            case PERCENT: return new PercentSplitStrategy();
            default: throw new IllegalArgumentException("Unknown split type: " + type);
        }
    }
}

enum SplitType { EQUAL, EXACT, PERCENT }
```

Expense

```
class Expense {
    private final String id;
    private final User paidBy;
    private final double amount;
    private final List<Split> splits;    // who owes what share of this expense

    Expense(String id, User paidBy, double amount, List<Split> splits) {
        this.id = id; this.paidBy = paidBy; this.amount = amount; this.splits = splits;
    }
    User getPaidBy() { return paidBy; }
    List<Split> getSplits() { return splits; }
}
```

BalanceSheet — the shared mutable state

```
class BalanceSheet {
    // balances[A][B] = net amount A owes B. Only one side is kept non-zero at a time.
    private final Map<User, Map<User, Double>> balances = new HashMap<>();

    // A owes B `amount` more (amount may be negative to reduce/reverse a debt).
    void adjust(User a, User b, double amount) {
        if (amount == 0) return;
        double aOwesB = get(a, b) - get(b, a) + amount;    // net after this adjustment
        if (aOwesB >= 0) {
            set(a, b, aOwesB);
            set(b, a, 0.0);
        } else {
            set(b, a, -aOwesB);
            set(a, b, 0.0);
        }
    }

    double get(User a, User b) {
        return balances.getOrDefault(a, Collections.emptyMap()).getOrDefault(b, 0.0);
    }

    private void set(User a, User b, double val) {
        balances.computeIfAbsent(a, k -> new HashMap<>()).put(b, val);
    }

    // Net position of a user across everyone: positive = is owed, negative = owes.
    double netBalance(User u, Set<User> everyone) {
        double net = 0;
        for (User other : everyone) {
            if (other.equals(u)) continue;
            net += get(other, u) - get(u, other);
        }
        return net;
    }
}
```

Group — where expenses are added and debts simplified

```
class Group {
    private final List<User> members = new ArrayList<>();
    private final BalanceSheet sheet = new BalanceSheet();
    private final List<Expense> expenses = new ArrayList<>();

    void addMember(User u) { members.add(u); }

    // Add an expense: validate participants, compute splits, update balances.
    // synchronized on `sheet` — the critical section is the multi-step balance
    // update, not the whole method, so we lock the sheet explicitly.
    Expense addExpense(User paidBy, double amount, List<Split> rawSplits, SplitType type) {
        if (!members.contains(paidBy)) {
            throw new NoSuchElementException("Payer is not a group member: " + paidBy);
        }
        for (Split s : rawSplits) {
            if (!members.contains(s.getUser())) {
                throw new NoSuchElementException("Unknown participant: " + s.getUser());
            }
        }
        List<Split> resolved = SplitStrategyFactory.get(type).compute(amount, rawSplits);

        Expense expense = new Expense(UUID.randomUUID().toString(), paidBy, amount, resolved)

        // Race: two threads adding expenses to the same group could both read a
        // balance, both compute a new value from the stale read, and one update
        // is lost. Synchronizing on the sheet serializes the read-modify-write.
        synchronized (sheet) {
            for (Split s : resolved) {
                if (s.getUser().equals(paidBy)) continue; // payer doesn't owe themselves
                sheet.adjust(s.getUser(), paidBy, s.getAmount());
            }
            expenses.add(expense);
        }
        return expense;
    }

    // Direct settle-up: reduce what `from` owes `to` by `amount`.
    void settle(User from, User to, double amount) {
        synchronized (sheet) {
            sheet.adjust(from, to, -amount);
        }
    }

    double owes(User a, User b) {
        synchronized (sheet) { return sheet.get(a, b); }
    }
}
```


Simplify debts — minimize the number of settle-up transactions

```
class DebtSimplifier {
    // Greedy: repeatedly match the biggest creditor with the biggest debtor.
    // Each match fully settles the smaller side, so at most (n-1) transactions
    // are produced for n people — provably minimal for net-settlement.
    static List<String> simplify(Group group, List<User> members, BalanceSheet sheet) {
        PriorityQueue<double[]> creditors = new PriorityQueue<>((x, y) -> Double.compare(y[0], x[0]));
        PriorityQueue<double[]> debtors = new PriorityQueue<>((x, y) -> Double.compare(y[0], x[0]));
        // (using parallel index arrays instead of double[] holding a User is a
        // simplification for illustration; real code pairs User with net amount)
        Map<User, Double> net = new HashMap<>();
        for (User u : members) net.put(u, sheet.netBalance(u, new HashSet<>(members)));

        PriorityQueue<Map.Entry<User, Double>> credit =
            new PriorityQueue<>((a, b) -> Double.compare(b.getValue(), a.getValue()));
        PriorityQueue<Map.Entry<User, Double>> debit =
            new PriorityQueue<>((a, b) -> Double.compare(a.getValue(), b.getValue()));
        for (Map.Entry<User, Double> e : net.entrySet()) {
            if (e.getValue() > 0.01) credit.add(e);
            else if (e.getValue() < -0.01) debit.add(e);
        }

        List<String> transactions = new ArrayList<>();
        while (!credit.isEmpty() && !debit.isEmpty()) {
            Map.Entry<User, Double> maxCred = credit.poll();
            Map.Entry<User, Double> maxDebt = debit.poll();
            double settled = Math.min(maxCred.getValue(), -maxDebt.getValue());

            transactions.add(maxDebt.getKey() + " pays " + maxCred.getKey() + " " + round2(settled));

            double remCred = maxCred.getValue() - settled;
            double remDebt = maxDebt.getValue() + settled;
            if (remCred > 0.01) credit.add(Map.entry(maxCred.getKey(), remCred));
            if (remDebt < -0.01) debit.add(Map.entry(maxDebt.getKey(), remDebt));
        }
        return transactions;
    }
}
```

Why lock the whole sheet and not per-pair? `addExpense` touches several pairs of users in one logical update (one payer, many participants); another thread reading balances mid-update would see a half-applied expense. Locking the shared `BalanceSheet` object keeps "one expense's balance effects are atomic" true with minimal code. For very high-traffic groups you could shard by group ID instead of locking one global sheet, but per-group locking is the right first answer.

Use-case walkthrough

1. `group.addMember(alice); group.addMember(bob); group.addMember(carol);`
2. Dinner, \$90, paid by Alice, split EQUAL among all three:
`addExpense(alice, 90, splits=[alice,bob,carol], EQUAL)`
 -> `EqualSplitStrategy`: each share = 30.0
 -> `sheet.adjust(bob, alice, 30); sheet.adjust(carol, alice, 30)`
 -> bob owes alice 30, carol owes alice 30
3. Taxi, \$40, paid by Bob, split EXACT: [alice:25, bob:15]
 -> `ExactSplitStrategy` validates $25+15 == 40$, ok
 -> `sheet.adjust(alice, bob, 25)`
 -> net: alice owes bob 25, but bob already owed alice 30
 -> `BalanceSheet.adjust` nets it: bob owes alice 5
4. `group.owes(bob, alice) -> 5.0` ; `group.owes(carol, alice) -> 30.0`
5. Bad input: splits [alice:20, bob:15] for a \$40 exact expense
 -> `ExactSplitStrategy` throws `IllegalArgumentException(... 35 ... 40)`
6. `DebtSimplifier.simplify(...)` on a bigger group collapses a chain of
 A->B->C debts into the minimum set of direct payments.

Extensions an interviewer may probe

- **Multi-currency groups** → `Expense` carries a `Currency` ; balances stored per currency pair, or converted via a pluggable `ExchangeRateProvider` .
 - **Recurring expenses / reminders** → an `Observer` that notifies members when a balance crosses a threshold or a recurring `Expense` is due.
 - **Partial settlements and payment history** → don't just zero balances on `settle` ; append a `Payment` record so the group has an auditable ledger, not just a live sheet.
-

7. Tic-Tac-Toe

The problem in one line. Design a two-player tic-tac-toe game on an $n \times n$ board that tracks turns, validates moves, and efficiently detects a win, loss, or draw after every move.

Step 1 — Clarify requirements

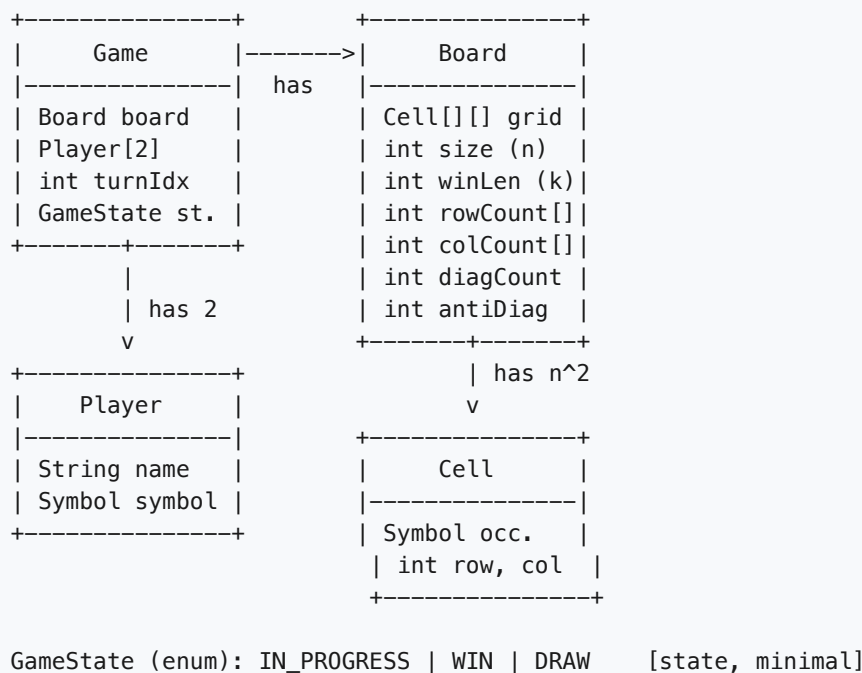
Questions worth asking out loud (and the assumptions this design makes):

- **Board size?** Classic is 3×3 , but design for a general $n \times n$ board — interviewers almost always ask "what if it's bigger?" *Build it in from the start.*
- **Win condition?** k -in-a-row (row, column, or diagonal). For 3×3 , $k = n = 3$; generalize so k can be smaller than n (e.g. 15×15 board, 5-in-a-row — Gomoku).
- **Players?** Exactly two, alternating turns, each with a fixed symbol (X/O).
- **How is a win detected?** Naively you'd rescan the whole board after every move — $O(n^2)$. We instead maintain running counts so each move is checked in $O(1)$.
- **Concurrency?** This is a local, turn-based, single-board game — one move happens at a time by construction (a player physically can't move out of turn). No two threads contend for the same cell, so no locking is needed. *Call this out explicitly — it shows you know when concurrency control is unnecessary, not just how to add it.*
- **Draw?** All cells filled with no winner.

Step 2 — Entities (the nouns)

Game, Board, Cell, Player, Symbol (X/O), Move, GameState (enum).

Step 3 — Relationships (class diagram)



Step 4 — Patterns chosen (and why)

- **Encapsulated running counters (Strategy-free optimization)** — the whole point of this problem is `Board` tracking per-row, per-column, and per-diagonal counts incrementally so a win can be checked in $O(1)$ after each move instead of $O(n^2)$ by rescanning. This is the detail interviewers are listening for.
- **Enum-based state** (`GameState`) instead of a full State-pattern class hierarchy — the transitions are simple (`IN_PROGRESS -> WIN` or `IN_PROGRESS -> DRAW`, both terminal), so a heavyweight State pattern would be over-engineering. Say this out loud: know when *not* to add a pattern.
- **Composition** — `Game` *has* a `Board` and *has* `Player` s; `Board` *has* `Cell` s. No inheritance needed since there's only one kind of board and one kind of player.
- **Factory-free construction** — symbols are just an enum with two values, so a factory would add no value here (contrast with Parking Lot's `VehicleFactory`, which has real branching logic).

Step 5 — Skeleton + key methods

Enums and value types

```
enum Symbol { X, O, EMPTY }
enum GameState { IN_PROGRESS, WIN, DRAW }

class Move {
    final int row, col;
    final Symbol symbol;
    Move(int row, int col, Symbol symbol) { this.row = row; this.col = col; this.symbol = sym
}

class Player {
    private final String name;
    private final Symbol symbol;
    Player(String name, Symbol symbol) { this.name = name; this.symbol = symbol; }
    public String getName() { return name; }
    public Symbol getSymbol() { return symbol; }
}
```

Cell — the unit of state

```
class Cell {
    private final int row, col;
    private Symbol occupant = Symbol.EMPTY;

    Cell(int row, int col) { this.row = row; this.col = col; }
    boolean isEmpty() { return occupant == Symbol.EMPTY; }
    void occupy(Symbol s) { this.occupant = s; }
    Symbol getOccupant() { return occupant; }
}
```

Board — $n \times n$, generalized to k -in-a-row, with $O(1)$ win check

The naive win check rescans the whole board after every move — $O(n^2)$, repeated for every move, so $O(n^4)$ over a full game. Instead, keep one running signed count per row, per column, and per diagonal. X contributes `+1` to every line through its cell, O contributes `-1`. A line hits `+winLength` (all X) or `-winLength` (all O) the instant it's won — so a win is checked in **$O(1)$ per move**, only touching the 4 counters affected by the last move. This only works cleanly for the classic "exactly one line matters, $k == n$ " case; for $k < n$ (Gomoku-style) each line is split into overlapping windows, so we fall back to a bounded local walk of length k around the move — still $O(k)$, never $O(n^2)$.

```

class Board {
    private final int size;                // n
    private final int winLength;           // k (k <= n); k == n for classic 3x3
    private final Cell[][] grid;

    // Signed running counts, valid when winLength == size (classic case):
    // +1 per X, -1 per O. A line is won the instant |count| == size.
    private final int[] rowCount;
    private final int[] colCount;
    private int diagCount;                // top-left -> bottom-right
    private int antiDiagCount;           // top-right -> bottom-left

    Board(int size, int winLength) {
        if (winLength > size) {
            throw new IllegalArgumentException("winLength cannot exceed board size");
        }
        this.size = size;
        this.winLength = winLength;
        this.grid = new Cell[size][size];
        for (int r = 0; r < size; r++) {
            for (int c = 0; c < size; c++) grid[r][c] = new Cell(r, c);
        }
        this.rowCount = new int[size];
        this.colCount = new int[size];
    }

    // Places a symbol and returns true if this move just completed a win.
    // 0(1): updates only the row/col/diagonal counters through (row, col).
    boolean place(int row, int col, Symbol symbol) {
        validateBounds(row, col);
        Cell cell = grid[row][col];
        if (!cell.isEmpty()) {
            throw new IllegalStateException("Cell (" + row + ", " + col + ") is already occupied");
        }
        cell.occupy(symbol);

        int delta = (symbol == Symbol.X) ? 1 : -1;
        rowCount[row] += delta;
        colCount[col] += delta;
        if (row == col) diagCount += delta;
        if (row + col == size - 1) antiDiagCount += delta;

        return Math.abs(rowCount[row]) == size
            || Math.abs(colCount[col]) == size
            || Math.abs(diagCount) == size
            || Math.abs(antiDiagCount) == size;
    }

    boolean isFull() {
        for (Cell[] row : grid) {
            for (Cell cell : row) if (cell.isEmpty()) return false;
        }
        return true;
    }

    private boolean inBounds(int r, int c) { return r >= 0 && r < size && c >= 0 && c < size; }

    private void validateBounds(int row, int col) {
        if (!inBounds(row, col)) {
            throw new IndexOutOfBoundsException("Move (" + row + ", " + col + ") is off the board");
        }
    }
}

```

```

    }
}
}

```

Why signed counters instead of a walk? For the classic case where the whole row/column must be filled by one symbol (`winLength == size`), a signed running sum is strictly $O(1)$ — no walking, no rescanning, just one integer add per affected line and a comparison against `size`. This is the answer interviewers are fishing for. The moment `winLength < size` (Gomoku's 5-in-a-row on a 15×15 board), a line can be won without being fully occupied, so the signed-sum trick no longer detects it alone — that's the natural point to mention the bounded local-walk fallback (Step 5 note in Extensions) without over-building it up front.

Game — orchestrates turns and state

```

class Game {
    private final Board board;
    private final Player[] players;    // exactly 2
    private int turnIndex = 0;
    private GameState state = GameState.IN_PROGRESS;

    Game(int size, int winLength, Player playerX, Player player0) {
        this.board = new Board(size, winLength);
        this.players = new Player[] { playerX, player0 };
    }

    // Plays one move for the current player. Throws if the game already ended
    // or the move is illegal; Board.place() throws for bounds/occupied cells.
    public GameState playMove(int row, int col) {
        if (state != GameState.IN_PROGRESS) {
            throw new IllegalStateException("Game already finished: " + state);
        }
        Player current = players[turnIndex];
        boolean won = board.place(row, col, current.getSymbol());

        if (won) {
            state = GameState.WIN;
        } else if (board.isFull()) {
            state = GameState.DRAW;
        } else {
            turnIndex = 1 - turnIndex;    // alternate turn; no lock needed (see Step 1)
        }
        return state;
    }

    public GameState getState() { return state; }
    public Player getCurrentPlayer() { return players[turnIndex]; }
}

```

Use-case walkthrough

```
1. Game game = new Game(3, 3, new Player("Alice", Symbol.X), new Player("Bob", Symbol.O));
2. game.playMove(0, 0); // Alice (X) -> IN_PROGRESS, turn passes to Bob
3. game.playMove(1, 1); // Bob (O) -> IN_PROGRESS, turn passes to Alice
4. game.playMove(0, 1); // Alice (X) -> IN_PROGRESS
5. game.playMove(2, 2); // Bob (O) -> IN_PROGRESS
6. game.playMove(0, 2); // Alice (X) completes row 0: X X X -> rowCount[0] becomes
                        // +3 == size -> WIN, no board rescan needed.
```

```
      0   1   2
+---+---+---+
0 | X | X | X |   <- winning row
+---+---+---+
1 |   | O |   |
+---+---+---+
2 |   |   | O |
+---+---+---+
```

```
7. game.playMove(1, 0); // -> throws IllegalStateException("Game already finished: WIN")
```

Extensions an interviewer may probe

- **Generalize to Gomoku (15×15, 5-in-a-row)** → `winLength < size` breaks the signed-sum trick (a line can win without being full), so swap `place()`'s win check for a bounded local walk of length `winLength` in each of the 4 directions around the last move — still $O(k)$ per move, never a full-board $O(n^2)$ rescan.
- **N players instead of 2** → widen `players[]`, cycle `turnIndex mod N`, and give each a distinct `Symbol`; the win check would need per-symbol counters instead of a signed X/O sum, but the $O(1)$ -per-move idea is unchanged.
- **Undo / move history** → record each `Move` in a stack on `Game`; undo simply reverses the same `delta` (`rowCount[row] -= delta`, etc.), which is exactly why the signed running sum is convenient — it's trivially reversible, unlike a recomputed rescan.

8. ATM Machine

The problem in one line. Design an ATM that lets a customer insert a card, authenticate with a PIN, and withdraw cash, deposit cash, or check balance — dispensing physical notes correctly and safely.

Step 1 — Clarify requirements

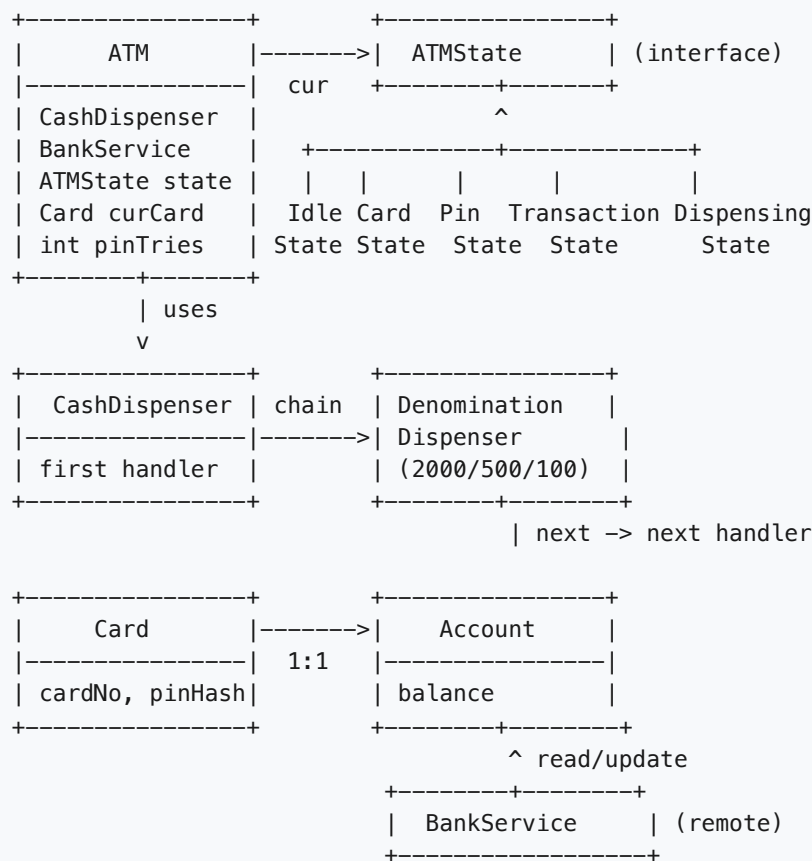
Questions worth asking out loud (and the assumptions this design makes):

- **Operations supported?** Withdraw, deposit, balance inquiry. Card and PIN required for all.
- **Who validates the PIN and account balance?** A remote `BankService` — the ATM is a dumb terminal that talks to the bank; it never owns the balance itself.
- **PIN attempts?** 3 tries, then the card is retained/blocked. *Confirm this — some variants just eject the card instead.*
- **Cash handling?** The ATM stocks fixed denominations (2000, 500, 100). Withdrawal amount must be exactly representable by available notes, or it's rejected.
- **Concurrency?** A single physical ATM serves one customer at a time end-to-end, but the cash inventory (denomination counts) can be read/audited concurrently by a monitoring thread — updates to it must be safe.
- **Failure modes?** Wrong PIN, insufficient funds in account, insufficient cash in machine, amount not dispensable by current notes, network/bank failure mid-transaction.

Step 2 — Entities (the nouns)

`ATM`, `Card`, `Account`, `BankService`, `CashDispenser`, `DenominationDispenser`, `Transaction` (+ `WithdrawTransaction` / `DepositTransaction` / `BalanceInquiryTransaction`), `ATMState` (+ concrete states).

Step 3 — Relationships (class diagram)



Transaction (abstract) <|-- Withdraw / Deposit / BalanceInquiry

Step 4 — Patterns chosen (and why)

- **State** for **ATMState** — the ATM's behavior (which buttons/actions are valid) changes completely depending on whether a card is inserted, a PIN is verified, or cash is being dispensed. Encoding this as `if/else` on an enum gets unmanageable fast; State lets each phase own its own transition logic.
- **Chain of Responsibility** for **CashDispenser** — dispensing ₹4700 means "as many 2000s as possible, then as many 500s as possible, then 100s." Each **DenominationDispenser** handles what it can of the remaining amount and forwards the rest down the chain, without the ATM needing to know the denomination ladder.
- **Strategy**-ish split for **Transaction** subtypes — withdraw/deposit/balance-inquiry each implement their own `execute`, so adding "mini statement" later doesn't touch the ATM.
- **Facade**: **BankService** is the single seam the ATM talks to for authentication and balance mutation — the ATM never touches the bank's database directly.

Step 5 — Skeleton + key methods

States (State pattern)

```
interface ATMState {
    void insertCard(ATM atm, Card card);
    void enterPin(ATM atm, int pin);
    void selectTransaction(ATM atm, Transaction txn);
    default void cancel(ATM atm) { atm.reset(); }
}

class IdleState implements ATMState {
    public void insertCard(ATM atm, Card card) {
        atm.setCurrentCard(card);
        atm.setState(new CardInsertedState());
    }
    public void enterPin(ATM atm, int pin) { throw new IllegalStateException("Insert card first"); }
    public void selectTransaction(ATM atm, Transaction txn) { throw new IllegalStateException("Insert card first"); }
}

class CardInsertedState implements ATMState {
    public void insertCard(ATM atm, Card card) { throw new IllegalStateException("Card already inserted"); }

    public void enterPin(ATM atm, int pin) {
        if (atm.getBankService().authenticate(atm.getCurrentCard(), pin)) {
            atm.resetPinTries();
            atm.setState(new AuthenticatedState());
        } else if (atm.incrementAndGetPinTries() >= ATM.MAX_PIN_TRIES) {
            atm.retainCard(); // too many wrong PINs -> keep the card
            atm.setState(new IdleState());
            throw new AuthenticationException("Max PIN attempts exceeded; card retained");
        } else {
            throw new AuthenticationException("Incorrect PIN, try again");
        }
    }

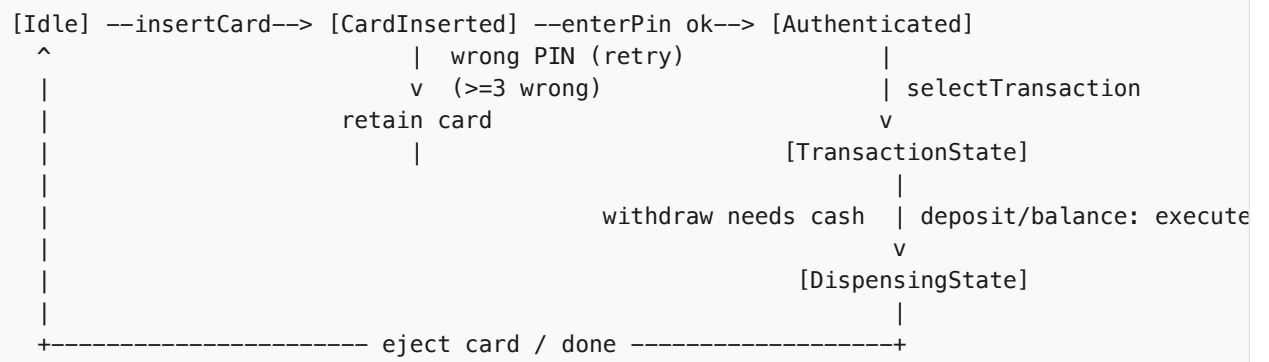
    public void selectTransaction(ATM atm, Transaction txn) { throw new IllegalStateException("Insert card first"); }
}

class AuthenticatedState implements ATMState {
    public void insertCard(ATM atm, Card card) { throw new IllegalStateException("Already authenticated"); }
    public void enterPin(ATM atm, int pin) { throw new IllegalStateException("Already authenticated"); }

    public void selectTransaction(ATM atm, Transaction txn) {
        atm.setState(new TransactionState());
        txn.execute(atm); // may itself move state -> DispensingState
        atm.setState(new IdleState()); // eject card, ready for next customer
    }
}

// Entered only for withdrawals, while notes are physically moving.
class DispensingState implements ATMState {
    public void insertCard(ATM atm, Card card) { throw new IllegalStateException("Busy dispensing"); }
    public void enterPin(ATM atm, int pin) { throw new IllegalStateException("Busy dispensing"); }
    public void selectTransaction(ATM atm, Transaction txn) { throw new IllegalStateException("Busy dispensing"); }
}
```

State transition diagram



Cash dispenser (Chain of Responsibility)

```
abstract class DenominationDispenser {
    protected DenominationDispenser next;
    protected final int denomination;
    protected int count; // notes currently stocked
    private final Object lock = new Object(); // guards `count` for this denomination

    DenominationDispenser(int denomination, int count) {
        this.denomination = denomination; this.count = count;
    }
    void setNext(DenominationDispenser next) { this.next = next; }

    // Dispense as much of `amount` as this denomination can, then pass the remainder on.
    int dispense(int amount) {
        int numNotes = 0;
        synchronized (lock) {
            numNotes = Math.min(amount / denomination, count);
            count -= numNotes; // safe decrement of shared inventory
        }
        int remaining = amount - numNotes * denomination;
        if (remaining > 0) {
            if (next == null) {
                // Nothing left in the chain but amount not fully covered -> roll back and fail
                synchronized (lock) { count += numNotes; }
                throw new UndispensableAmountException("Cannot dispense amount with available");
            }
            try {
                next.dispense(remaining);
            } catch (InsufficientCashException | UndispensableAmountException e) {
                synchronized (lock) { count += numNotes; } // roll back this link too
                throw e;
            }
        }
        return numNotes;
    }
}

class TwoThousandDispenser extends DenominationDispenser { TwoThousandDispenser(int c) { super(c, 10); }
class FiveHundredDispenser extends DenominationDispenser { FiveHundredDispenser(int c) { super(c, 20); }
class HundredDispenser extends DenominationDispenser { HundredDispenser(int c) { super(c, 100); }

class CashDispenser {
    private final DenominationDispenser chain; // head of chain: 2000 -> 500 -> 100

    CashDispenser(DenominationDispenser chain) { this.chain = chain; }

    // Throws if the machine doesn't have enough total cash or can't make exact change.
    void dispense(int amount) {
        if (amount % 100 != 0) throw new UndispensableAmountException("Amount must be multiple of 100");
        chain.dispense(amount);
    }
}
```

Bank-side entities and transactions

```
class Card {
    private final String cardNo;
    private final String accountId;
    Card(String cardNo, String accountId) { this.cardNo = cardNo; this.accountId = accountId; }
    String getAccountId() { return accountId; }
}

interface BankService {
    boolean authenticate(Card card, int pin);
    double getBalance(String accountId);
    void withdraw(String accountId, double amount);    // throws InsufficientFundsException
    void deposit(String accountId, double amount);
}

abstract class Transaction {
    protected final double amount;
    Transaction(double amount) { this.amount = amount; }
    abstract void execute(ATM atm);
}

class WithdrawTransaction extends Transaction {
    WithdrawTransaction(double amount) { super(amount); }

    void execute(ATM atm) {
        String accId = atm.getCurrentCard().getAccountId();
        // 1. Ask the bank first -- never dispense cash before the account is actually debited
        atm.getBankService().withdraw(accId, amount);    // throws InsufficientFundsException
        atm.setState(new DispensingState());
        try {
            atm.getCashDispenser().dispense((int) amount); // throws Undispensable/Insufficient
        } catch (RuntimeException e) {
            atm.getBankService().deposit(accId, amount);    // compensate: refund on dispense
            throw e;
        }
    }
}

class DepositTransaction extends Transaction {
    DepositTransaction(double amount) { super(amount); }
    void execute(ATM atm) {
        atm.getBankService().deposit(atm.getCurrentCard().getAccountId(), amount);
    }
}

class BalanceInquiryTransaction extends Transaction {
    BalanceInquiryTransaction() { super(0); }
    void execute(ATM atm) {
        double bal = atm.getBankService().getBalance(atm.getCurrentCard().getAccountId());
        atm.display("Balance: " + bal);
    }
}
```

The ATM — orchestrator

```
class ATM {
    static final int MAX_PIN_TRIES = 3;

    private ATMState state = new IdleState();
    private final CashDispenser cashDispenser;
    private final BankService bankService;
    private Card currentCard;
    private int pinTries = 0;

    ATM(CashDispenser cashDispenser, BankService bankService) {
        this.cashDispenser = cashDispenser; this.bankService = bankService;
    }

    // Public API delegates to the current state -- callers never branch on state themselves.
    public void insertCard(Card card) { state.insertCard(this, card); }
    public void enterPin(int pin) { state.enterPin(this, pin); }
    public void selectTransaction(Transaction txn) { state.selectTransaction(this, txn); }

    void setState(ATMState s) { this.state = s; }
    void reset() { this.currentCard = null; this.pinTries = 0; this.state = new IdleState(); }
    void retainCard() { this.currentCard = null; } // card kept inside the machine
    int incrementAndGetPinTries() { return ++pinTries; }
    void resetPinTries() { pinTries = 0; }

    Card getCurrentCard() { return currentCard; }
    void setCurrentCard(Card c) { this.currentCard = c; }
    CashDispenser getCashDispenser() { return cashDispenser; }
    BankService getBankService() { return bankService; }
    void display(String msg) { System.out.println(msg); }
}
```

Why this is safe under concurrency. A single ATM only ever serves one customer, so `ATMState` transitions need no locking. The part that *is* shared is the physical cash inventory: an ops thread might be reading counts for a "cash low" alert while a customer is withdrawing. Each `DenominationDispenser` guards its own `count` with a private lock and does a read-decrement-rollback under that lock, so two concurrent withdrawals never double-spend the same notes. Account balance mutation is delegated entirely to `BankService` — the ATM never mutates `balance` locally, so the bank's own transactional guarantees (not the ATM's) protect against double-withdrawal on the money side. The withdraw flow also debits the bank *before* dispensing and refunds on a dispense failure, so a machine that runs out of physical cash never leaves the customer's account permanently short.

Use-case walkthrough

1. atm.insertCard(card) → Idle → CardInsertedState
2. atm.enterPin(4321) → wrong once (pinTries=1), correct on retry
→ CardInsertedState → AuthenticatedState
3. atm.selectTransaction(new WithdrawTransaction(4700))
→ AuthenticatedState → TransactionState
→ bankService.withdraw(acc, 4700) [bank debits first]
→ TransactionState → DispensingState
→ chain: 2000-dispenser gives 2, remaining 700
→ 500-dispenser gives 1, remaining 200
→ 100-dispenser gives 2, remaining 0
→ DispensingState → IdleState, card ejected
4. Next customer withdraws 4750
→ UndispensableAmountException (no way to make 50 from 2000/500/100)
→ caught by the ATM UI layer, card ejected, no bank debit occurred

Extensions an interviewer may probe

- **Multiple ATMs sharing a cash-refill schedule** → move `CashDispenser` inventory into a persisted store polled/updated by an ops service instead of in-memory counts.
 - **Contactless / mobile-initiated withdrawal** → add a new entry state that skips `CardInsertedState` but still routes through `AuthenticatedState`.
 - **Receipt printing / mini statement** → new `Transaction` subtype; ATM and State classes stay untouched (Open/Closed).
-

9. Logging Framework

The problem in one line. Design an in-process logging framework — like a mini Log4j — that lets callers log messages at a level (DEBUG/INFO/WARN/ERROR), routes each message to one or more sinks (console, file), and stays correct when many threads log at once.

Step 1 — Clarify requirements

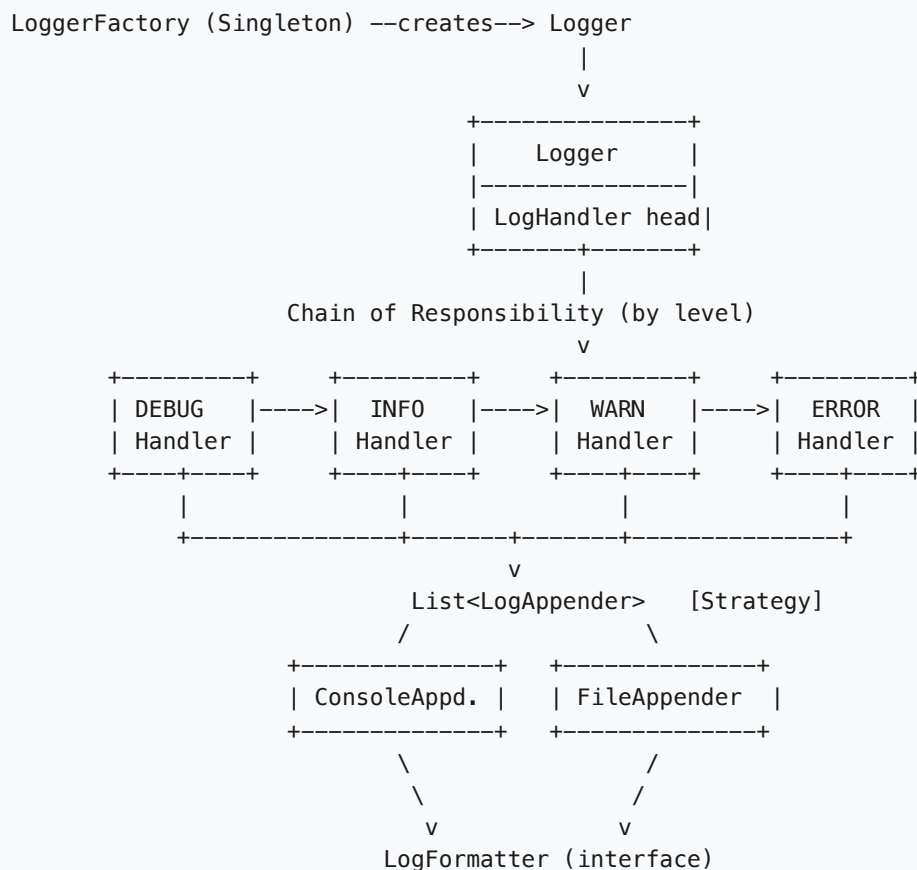
Questions worth asking out loud (and the assumptions this design makes):

- **Levels?** DEBUG < INFO < WARN < ERROR, ordered by severity. A logger has a configured **minimum level**; messages below it are dropped.
- **Where does output go?** Multiple destinations at once — console and a rotating file are the two this design covers. *Confirm whether both must always fire, or just one.*
- **Formatting?** The line layout (timestamp, level, thread, message) should be swappable without touching the sinks — pluggable formatter.
- **Concurrency?** Many threads call `logger.info(...)` concurrently. Two threads writing to the same `FileWriter` at once must not interleave bytes or corrupt the file.
- **Failure isolation?** If the file sink can't write (disk full, permissions), the app must keep running — logging failures are never allowed to crash business logic.
- **One logger instance or many?** A single global entry point (`LoggerFactory`) hands out loggers so config (level, appenders) is set in one place.

Step 2 — Entities (the nouns)

`LogMessage` , `LogLevel` , `LogHandler` (chain link per level), `LogAppender` (sink), `LogFormatter` , `Logger` , `LoggerFactory` .

Step 3 — Relationships (class diagram)



Step 4 — Patterns chosen (and why)

- **Chain of Responsibility** for level handling — each `LogHandler` owns one level, decides whether *its* level accepts the message, and always passes it further down the chain so a higher-severity handler downstream can also see it. This mirrors how Log4j/SLF4J walk a level hierarchy without one giant `if/else`.
- **Strategy** for `LogAppender` — console vs. file (vs. future: network, DB) are interchangeable sinks behind one interface. Same idea for `LogFormatter` — swap the line layout without touching appenders.
- **Singleton** for `LoggerFactory` — one place configures level + appenders and hands out the (possibly cached) `Logger` for a class name, exactly like `LoggerFactory.getLogger(...)` in real frameworks.
- **Producer/consumer queue** (mentioned as an extension) for async logging — decouples the calling thread from slow I/O.

Step 5 — Skeleton + key methods

Level, message, and formatter

```
enum LogLevel { DEBUG, INFO, WARN, ERROR }    // ordinal = severity order

class LogMessage {
    final LogLevel level;
    final String text;
    final long timestampMs;
    final String threadName;

    LogMessage(LogLevel level, String text) {
        this.level = level;
        this.text = text;
        this.timestampMs = System.currentTimeMillis();
        this.threadName = Thread.currentThread().getName();
    }
}

interface LogFormatter {
    String format(LogMessage msg);
}

// Default layout: "[time] [LEVEL] [thread] message"
class SimpleFormatter implements LogFormatter {
    public String format(LogMessage msg) {
        return String.format("[%d] [%s] [%s] %s",
            msg.timestampMs, msg.level, msg.threadName, msg.text);
    }
}
```

Appenders — pluggable sinks, thread-safe, fail without crashing

```
interface LogAppender {
    void append(String formattedLine);
}

// Console: cheap, but still must not interleave lines from two threads.
class ConsoleAppender implements LogAppender {
    private final Object lock = new Object();

    public void append(String line) {
        synchronized (lock) { // one line at a time on stdout
            System.out.println(line);
        }
    }
}

// File: the real race — two threads calling write() on the same stream
// at once can corrupt output or lose bytes. synchronized serializes writes.
// A failed write (disk full, permission denied) is caught so the caller's
// business logic never crashes because logging failed.
class FileAppender implements LogAppender {
    private final Object lock = new Object();
    private final Writer writer;

    FileAppender(String path) throws IOException {
        this.writer = new BufferedWriter(new FileWriter(path, true)); // append mode
    }

    public void append(String line) {
        synchronized (lock) {
            try {
                writer.write(line);
                writer.write(System.lineSeparator());
                writer.flush(); // durability over raw throughput
            } catch (IOException e) {
                // Swallow + report to stderr only — a broken sink must not
                // take down the app. A real system would also flip a
                // "sink unhealthy" flag and alert.
                System.err.println("FileAppender failed: " + e.getMessage());
            }
        }
    }
}
```

Chain of Responsibility — one handler per level

```
abstract class LogHandler {
    protected LogHandler next;                // next link in the chain
    protected final LogLevel level;           // level this handler owns
    protected final List<LogAppender> appenders;
    protected final LogFormatter formatter;

    LogHandler(LogLevel level, List<LogAppender> appenders, LogFormatter formatter) {
        this.level = level;
        this.appenders = appenders;
        this.formatter = formatter;
    }

    LogHandler setNext(LogHandler next) { this.next = next; return next; }

    // Every handler that is >= the message's level writes it, then the
    // message still flows down so lower-in-chain (higher-severity)
    // handlers can also act on it (e.g. ERROR could also page on-call).
    void handle(LogMessage msg, LogLevel minLevel) {
        if (msg.level.ordinal() >= minLevel.ordinal() && msg.level == this.level) {
            writeToAllAppenders(msg);
        }
        if (next != null) next.handle(msg, minLevel);
    }

    private void writeToAllAppenders(LogMessage msg) {
        String line = formatter.format(msg);
        for (LogAppender appender : appenders) {
            appender.append(line);              // each appender independently thread-safe
        }
    }
}

class DebugHandler extends LogHandler {
    DebugHandler(List<LogAppender> a, LogFormatter f) { super(LogLevel.DEBUG, a, f); }
}
class InfoHandler extends LogHandler {
    InfoHandler(List<LogAppender> a, LogFormatter f) { super(LogLevel.INFO, a, f); }
}
class WarnHandler extends LogHandler {
    WarnHandler(List<LogAppender> a, LogFormatter f) { super(LogLevel.WARN, a, f); }
}
class ErrorHandler extends LogHandler {
    ErrorHandler(List<LogAppender> a, LogFormatter f) { super(LogLevel.ERROR, a, f); }
}
```

Logger and Singleton factory

```
class Logger {
    private final LogHandler chainHead;
    private final LogLevel minLevel;           // messages below this are dropped entirely

    Logger(LogHandler chainHead, LogLevel minLevel) {
        this.chainHead = chainHead;
        this.minLevel = minLevel;
    }

    public void debug(String text) { log(LogLevel.DEBUG, text); }
    public void info(String text)  { log(LogLevel.INFO, text); }
    public void warn(String text)  { log(LogLevel.WARN, text); }
    public void error(String text) { log(LogLevel.ERROR, text); }

    private void log(LogLevel level, String text) {
        if (level.ordinal() < minLevel.ordinal()) return; // cheap early exit, no I/O
        chainHead.handle(new LogMessage(level, text), minLevel);
    }
}

// Singleton: one configured entry point for the whole app, like
// LoggerFactory.getLogger(...) in Log4j/SLF4J.
class LoggerFactory {
    private static final LoggerFactory INSTANCE = new LoggerFactory();
    public static LoggerFactory getInstance() { return INSTANCE; }

    private final Map<String, Logger> cache = new ConcurrentHashMap<>();
    private volatile LogLevel globalMinLevel = LogLevel.INFO;
    private volatile List<LogAppender> globalAppenders = new ArrayList<>();
    private LogFormatter formatter = new SimpleFormatter();

    private LoggerFactory() { }

    public void configure(LogLevel minLevel, List<LogAppender> appenders, LogFormatter format
        this.globalMinLevel = minLevel;
        this.globalAppenders = appenders;
        this.formatter = formatter;
        cache.clear();           // re-build loggers with new config
    }

    // One logger per class name, built once, reused on every call.
    public Logger getLogger(String name) {
        return cache.computeIfAbsent(name, n -> buildLogger());
    }

    private Logger buildLogger() {
        LogHandler debug = new DebugHandler(globalAppenders, formatter);
        LogHandler info  = new InfoHandler(globalAppenders, formatter);
        LogHandler warn  = new WarnHandler(globalAppenders, formatter);
        LogHandler error = new ErrorHandler(globalAppenders, formatter);
        debug.setNext(info).setNext(warn).setNext(error); // build the chain
        return new Logger(debug, globalMinLevel);
    }
}
```

Where's the real race, and why is it safe? Every thread that logs eventually calls `appender.append(...)`. Two threads racing on `FileAppender.append` could otherwise interleave

partial writes on the same underlying `Writer`. `synchronized (lock)` inside each appender turns "many threads write" into "many threads queue up to write one at a time" — correctness is enforced at the sink, not by locking the whole `Logger`, so unrelated appenders (console vs. file) never block each other.

Use-case walkthrough

1. `LoggerFactory.getInstance().configure(LogLevel.INFO, List.of(new ConsoleAppender(), new FileAppender("/var/log/app.log")), new SimpleFormatter());`
2. `Logger log = LoggerFactory.getInstance().getLogger("OrderService");`
3. Thread A: `log.debug("computing total")`
→ dropped in `Logger.log()` before touching the chain (`DEBUG < INFO`)
4. Thread B (concurrently): `log.info("order placed: 123")`
→ passes `minLevel` check → `chainHead.handle(...)`
→ `DebugHandler`: level mismatch, passes to next
→ `InfoHandler`: matches → writes to `ConsoleAppender` + `FileAppender` (each appender synchronizes its own write; A and B never interleave)
→ passes to `WarnHandler`, `ErrorHandler`: level mismatch, no-op
5. `FileAppender`'s disk fills up on the next write
→ `IOException` is caught inside `FileAppender.append()`
→ `stderr` gets a one-line failure note; `OrderService` keeps running

Extensions an interviewer may probe

- **Async logging** → calling thread pushes `LogMessage`s onto a `BlockingQueue`; a single dedicated writer thread drains it and calls the appenders. Removes I/O latency from request threads; trade-off is messages can be lost on a hard crash before the queue drains.
 - **Log rotation** → `FileAppender` checks file size/date on each write and rolls over to `app.log.1`, `app.log.2`, ... — an internal detail, same `LogAppender` interface.
 - **Per-logger levels/appenders** → `LoggerFactory` already keys loggers by name; extend `configure` to accept overrides per package/class prefix, same lookup pattern as real frameworks (`com.foo` more verbose than `root`).
-

10. Rate Limiter

The problem in one line. Design a rate limiter that caps how many requests each client can make in a given time window, is safe under heavy concurrent access, and supports swapping the underlying throttling algorithm.

Step 1 — Clarify requirements

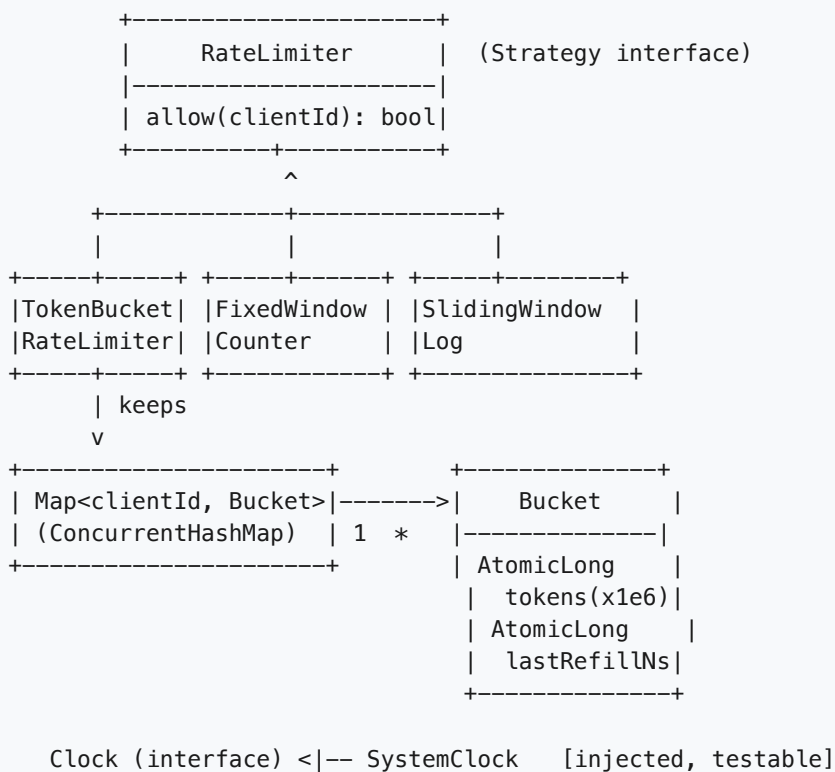
Questions worth asking out loud (and the assumptions this design makes):

- **Per what?** Per `clientId` (API key, user ID, or IP) — every client gets an independent budget. *Confirm the key; this design keys on a `String clientId`.*
- **What rule?** e.g. "100 requests per second, burst up to 100." Token bucket is the default answer because it allows bursts up to capacity and then smooths out — closer to real API gateways (AWS, Stripe) than a naive fixed window.
- **What happens on violation?** Reject the request (return `false` / a boolean decision) rather than throw, so the caller's hot path isn't paying exception cost — but expose a `RateLimitExceededException` variant too, since interviewers often want to see both discussed.
- **Distributed or single-node?** Single JVM in-memory for this design (a `ConcurrentHashMap` of buckets). *Call out that a distributed version needs Redis + Lua/atomic scripts — same interface, different `RateLimiter` implementation.*
- **Concurrency?** Many threads call `allow(clientId)` at once for the same client (e.g. a hot API key). Token consumption must never let two threads both "spend" the last token — that is the core of this problem.

Step 2 — Entities (the nouns)

`RateLimiter` (interface), `TokenBucket`, `FixedWindowCounter`, `SlidingWindowLog`, `Bucket` (per-client state), `Clock`, `RateLimiterRegistry`.

Step 3 — Relationships (class diagram)



Step 4 — Patterns chosen (and why)

- **Strategy** for `RateLimiter` — the throttling algorithm (token bucket, fixed window, sliding log) is swappable behind one interface; callers never know which is active.
- **Registry / per-key state** — a `ConcurrentHashMap<String, Bucket>` gives each client its own independent limiter state without a global lock across clients.
- **Dependency Injection** for `Clock` — time is passed through an interface (`Clock.nanoTime()`) instead of calling `System.nanoTime()` directly, so tests can advance time deterministically without `Thread.sleep`.
- **Immutable config, mutable state split** — capacity and refill rate are `final`; only the token count and last-refill timestamp mutate, and they mutate through atomics, not locks, everywhere it's safe to do so.

Step 5 — Skeleton + key methods

The Strategy interface and Clock abstraction

```
interface RateLimiter {
    // Returns true if the request is allowed (and consumes quota), false if throttled.
    boolean allow(String clientId);
}

interface Clock {
    long nowNanos();
}

class SystemClock implements Clock {
    public long nowNanos() { return System.nanoTime(); }
}
```

Comparing the three algorithms (why token bucket wins)

- **FixedWindowCounter** — count resets every window (e.g. every second). Simple, $O(1)$, but has the classic **boundary burst problem**: a client can fire `limit` requests at `t=0.999s` and another `limit` at `t=1.001s`, i.e. `2x limit` in `2ms`, because the counter resets exactly on the boundary instead of tracking a rolling window.
- **SlidingWindowLog** — keep a timestamp per accepted request, evict ones older than the window, allow if `count < limit`. Exact, no boundary burst, but $O(n)$ memory per client and $O(n)$ per check — expensive under load.
- **TokenBucket (primary)** — a bucket holds up to `capacity` tokens, refilled continuously at `refillRate` tokens/sec. Each request spends one token; no token, no request. It allows short bursts up to `capacity` (good for real traffic) then throttles to a smooth steady rate — no reset-boundary cliff, $O(1)$ memory and $O(1)$ check per client. This is what AWS, Stripe, and most API gateways actually use, so it's the expected primary answer.

Token bucket, capacity = 5, refill = 1 token / 200ms:

```
fill -> [#####] full (5/5)
3 requests arrive at once:
      [##...] 2/5 left, 3 tokens spent instantly (burst allowed)
200ms pass, lazy refill adds 1 token:
      [###...] 3/5
... continues refilling toward capacity, never exceeding it ...
```

Bucket — the per-client, thread-safe unit of state

```
class Bucket {
    private final long capacityMicroTokens;    // capacity * 1_000_000, for fractional prec
    private final long refillMicroTokensPerNano; // refillRate expressed per nanosecond
    private final AtomicLong microTokens;      // current tokens, scaled by 1e6
    private final AtomicLong lastRefillNanos;

    Bucket(long capacity, double refillPerSecond, long nowNanos) {
        this.capacityMicroTokens = capacity * 1_000_000L;
        // scale so refill math stays in integers -> avoids float drift under contention
        this.refillMicroTokensPerNano = (long) ((refillPerSecond * 1_000_000L) / 1_000_000_000L);
        this.microTokens = new AtomicLong(capacityMicroTokens); // start full
        this.lastRefillNanos = new AtomicLong(nowNanos);
    }

    // The heart of the design: lazily refill based on elapsed time, then try to
    // atomically spend one token. Must be race-free under many concurrent callers.
    boolean tryConsume(long nowNanos) {
        refill(nowNanos);
        // CAS loop: two threads racing on the "last token" must not both succeed.
        // Without this loop, thread A and B could both read tokens=1, both decide
        // "I can go", and both decrement -> tokens go negative and the limit is broken.
        while (true) {
            long current = microTokens.get();
            if (current < 1_000_000L) { // less than one whole token available
                return false; // reject: caller is over budget
            }
            long updated = current - 1_000_000L;
            if (microTokens.compareAndSet(current, updated)) {
                return true; // won the race, token spent
            }
            // else: another thread changed it first -> retry with fresh value
        }
    }

    private void refill(long nowNanos) {
        long prev = lastRefillNanos.get();
        long elapsed = nowNanos - prev;
        if (elapsed <= 0) return;
        // Only one thread should "commit" a given refill interval; CAS the timestamp
        // first so concurrent callers don't all add tokens for the same elapsed slice.
        if (lastRefillNanos.compareAndSet(prev, nowNanos)) {
            long minted = elapsed * refillMicroTokensPerNano;
            if (minted <= 0) return;
            microTokens.getAndUpdate(t -> Math.min(capacityMicroTokens, t + minted));
        }
        // If the CAS lost, another thread already advanced lastRefillNanos for this
        // slice; that thread's getAndUpdate will account for the elapsed time instead.
    }
}
```

Why not just `synchronized tryConsume()` ? It would be correct and simpler, and is a perfectly good answer to lead with. The atomic/CAS version is what you upgrade to when the interviewer pushes on throughput: `synchronized` serializes *all* callers for one client behind a single lock, while the CAS loop only retries the (rare) callers who actually collided, so a hot client scales better across cores. Either is acceptable — say which one you're choosing and why.

TokenBucket rate limiter and the per-client registry

```
class TokenBucketRateLimiter implements RateLimiter {
    private final long capacity;
    private final double refillPerSecond;
    private final Clock clock;
    private final ConcurrentHashMap<String, Bucket> buckets = new ConcurrentHashMap<>();

    TokenBucketRateLimiter(long capacity, double refillPerSecond, Clock clock) {
        this.capacity = capacity;
        this.refillPerSecond = refillPerSecond;
        this.clock = clock;
    }

    public boolean allow(String clientId) {
        if (clientId == null || clientId.isEmpty()) {
            throw new IllegalArgumentException("clientId must not be blank");
        }
        long now = clock.nowNanos();
        // computeIfAbsent is atomic per-key: two threads creating the SAME client's
        // first bucket will not create two Buckets and silently double the quota.
        Bucket bucket = buckets.computeIfAbsent(clientId, id -> new Bucket(capacity, refillPe
        return bucket.tryConsume(now);
    }
}

// Alternative surface for callers who want a thrown error instead of a boolean –
// same underlying limiter, different failure contract at the edge (e.g. a servlet filter
// that wants to short-circuit with a 429 via exception handling middleware).
class ThrowingRateLimiter {
    private final RateLimiter delegate;
    ThrowingRateLimiter(RateLimiter delegate) { this.delegate = delegate; }

    void checkAllowed(String clientId) {
        if (!delegate.allow(clientId)) {
            throw new RateLimitExceededException("Rate limit exceeded for " + clientId);
        }
    }
}

class RateLimitExceededException extends RuntimeException {
    RateLimitExceededException(String msg) { super(msg); }
}
```

FixedWindowCounter (for comparison, shown briefly)

```
class FixedWindowCounter implements RateLimiter {
    private final long limit;
    private final long windowNanos;
    private final Clock clock;
    private final ConcurrentHashMap<String, long[]> windows = new ConcurrentHashMap<>();
    // long[] {windowStartNanos, count} – packed together so one map lookup gets both

    FixedWindowCounter(long limit, long windowNanos, Clock clock) {
        this.limit = limit; this.windowNanos = windowNanos; this.clock = clock;
    }

    public synchronized boolean allow(String clientId) {
        long now = clock.nowNanos();
        long[] w = windows.computeIfAbsent(clientId, id -> new long[]{now, 0});
        if (now - w[0] >= windowNanos) { // window elapsed: reset (this is the burst edge)
            w[0] = now;
            w[1] = 0;
        }
        if (w[1] >= limit) return false;
        w[1]++;
        return true;
    }
}
```

Use-case walkthrough

1. `RateLimiter limiter = new TokenBucketRateLimiter(5, 5.0, new SystemClock());`
→ capacity 5 tokens, refills at 5 tokens/sec (1 every 200ms)
2. Client "api-key-42" fires 5 requests instantly at $t=0$:
→ `allow()` x5: each `tryConsume()` CAS-decrements; all 5 succeed (burst spent)
3. 6th request arrives 10ms later, still $t \approx 0$:
→ `refill()` mints ~ 0 new tokens ($10\text{ms} * 5/\text{sec} \approx 0.05$ tokens, rounds down)
→ `tokens < 1_000_000` microTokens → `tryConsume()` returns false → caller rejected
4. 250ms later, request arrives:
→ `refill()` mints $250\text{ms} * 5/\text{sec} = 1.25$ tokens → bucket now has ~ 1 token
→ `tryConsume()` succeeds, spends it, leaves ~ 0.25 tokens
5. 50 threads hit the SAME client concurrently when 1 token remains:
→ all 50 call `tryConsume()` → all read `current=1_000_000` in the CAS loop
→ exactly one thread's `compareAndSet` wins and returns true
→ the other 49 retry the loop, re-read `tokens=0`, and return false
→ no double-spend, no negative token count, regardless of thread count

Extensions an interviewer may probe

- **Distributed rate limiting across many app servers** → swap `ConcurrentHashMap<String, Bucket>` for Redis-backed buckets updated via an atomic Lua script (`INCR` + `PEXPIRE`), same `RateLimiter` interface — the Strategy boundary absorbs the change.

- **Different limits per tier (free vs. paid)** → `Bucket` construction reads `capacity / refillRate` from a per-client config lookup instead of one fixed pair for everyone.
 - **Sliding window without the $O(n)$ memory cost** → sliding window *counter* (two fixed windows weighted by overlap) approximates `SlidingWindowLog` accuracy at `TokenBucket`-like $O(1)$ cost; worth mentioning as the "best of both" answer.
-

11. Notification Service

The problem in one line. Design a notification service that sends a message to a user through the channels they've opted into (email, SMS, push), formats it from a template, and delivers it asynchronously without letting one slow channel block the others.

Step 1 — Clarify requirements

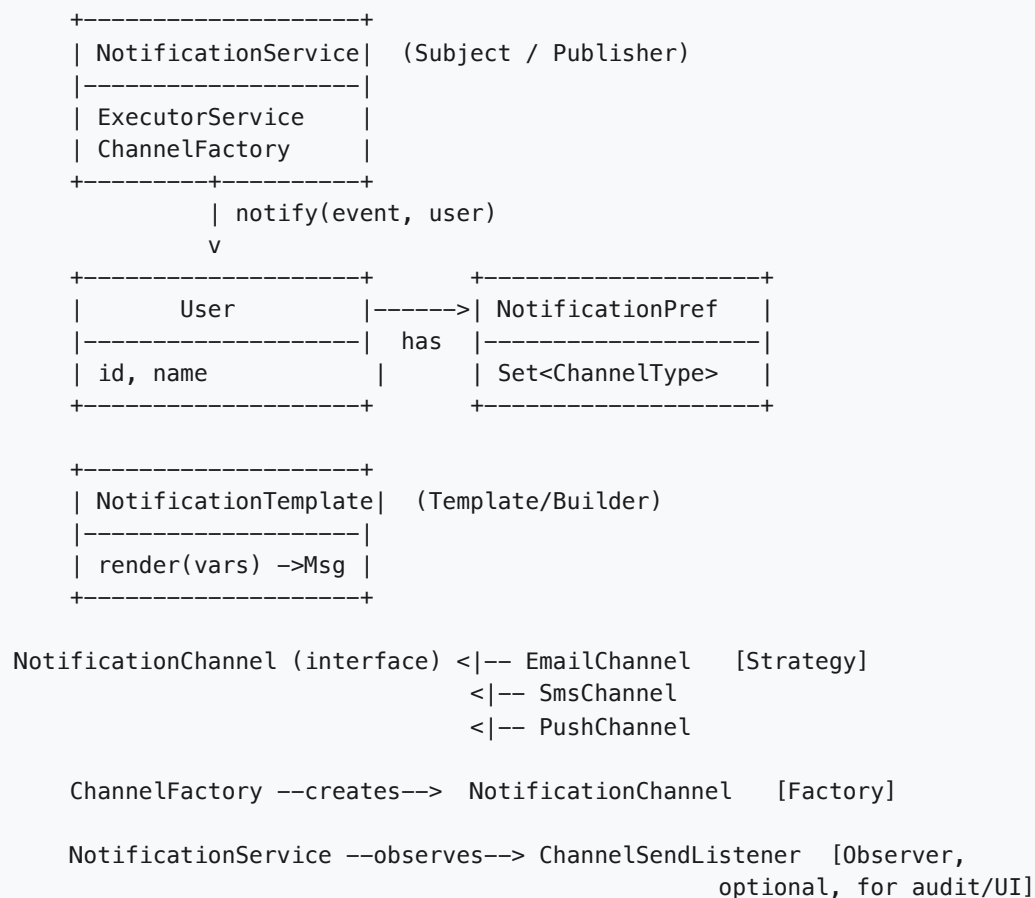
Questions worth asking out loud (and the assumptions this design makes):

- **Channels?** Email, SMS, Push to start — pluggable so a new channel (e.g. Slack, WhatsApp) can be added without touching dispatch logic.
- **Per-user preferences?** Yes — a user opts into a subset of channels; a channel with no subscribed users for an event does nothing.
- **Templating?** Messages are built from a `NotificationTemplate` (subject/body with placeholders) rather than hand-built strings at each call site.
- **Delivery guarantee?** Best-effort with retries — a transient gateway failure on one channel must not fail or block the others. *Confirm this — exactly-once delivery would need a durable queue, out of scope here.*
- **Concurrency?** Sends fan out to multiple channels per notification; each should run concurrently (a slow SMS gateway must not delay email/push), and multiple notifications may be in flight at once.

Step 2 — Entities (the nouns)

`User`, `NotificationPreference`, `NotificationTemplate`, `Notification`, `NotificationChannel` (+
`Email / Sms / Push`), `ChannelFactory`, `NotificationService`.

Step 3 — Relationships (class diagram)



Step 4 — Patterns chosen (and why)

- **Strategy + Factory** for **NotificationChannel** — sending differs per channel (SMTP vs. SMS gateway vs. push API); **ChannelFactory** hides the concrete class from the service so adding a channel means adding one class, not editing dispatch code.
- **Observer** — subscribers (e.g. an audit log, a delivery-status UI) register on the **NotificationService** and are told when a send succeeds or fails, without the service knowing who's listening.
- **Template** for **NotificationTemplate** — separates *what* the message says from *how* it's delivered; the same template renders once and is handed to every channel.
- **Thread pool (Executor)** for dispatch — decouples "decide who to notify" from "wait on a network call," so channels run concurrently instead of serially.

Step 5 — Skeleton + key methods

Channel hierarchy (Strategy) and Factory

```
interface NotificationChannel {
    ChannelType getType();
    // Throws on failure; caller (dispatcher) decides retry policy.
    void send(String to, Message message) throws NotificationException;
}

enum ChannelType { EMAIL, SMS, PUSH }

class EmailChannel implements NotificationChannel {
    public ChannelType getType() { return ChannelType.EMAIL; }
    public void send(String to, Message m) throws NotificationException {
        // ... call SMTP/email API; wrap any IOException as NotificationException
    }
}

class SmsChannel implements NotificationChannel {
    public ChannelType getType() { return ChannelType.SMS; }
    public void send(String to, Message m) throws NotificationException {
        // ... call SMS gateway; can be the slow one – must not block others
    }
}

class PushChannel implements NotificationChannel {
    public ChannelType getType() { return ChannelType.PUSH; }
    public void send(String to, Message m) throws NotificationException {
        // ... call push provider (FCM/APNs)
    }
}

// Factory: dispatcher asks for a channel by type, never `new`s one directly.
class ChannelFactory {
    private final Map<ChannelType, NotificationChannel> channels = new HashMap<>();
    ChannelFactory() {
        channels.put(ChannelType.EMAIL, new EmailChannel());
        channels.put(ChannelType.SMS, new SmsChannel());
        channels.put(ChannelType.PUSH, new PushChannel());
    }
    NotificationChannel get(ChannelType type) {
        NotificationChannel c = channels.get(type);
        if (c == null) throw new IllegalArgumentException("No channel for " + type);
        return c;
    }
}
```

Template (Template/Builder) and Preferences

```
class Message {
    final String subject;
    final String body;
    Message(String subject, String body) { this.subject = subject; this.body = body; }
}

class NotificationTemplate {
    private final String id;
    private final String subjectPattern;    // e.g. "Order {orderId} shipped"
    private final String bodyPattern;

    NotificationTemplate(String id, String subjectPattern, String bodyPattern) {
        this.id = id; this.subjectPattern = subjectPattern; this.bodyPattern = bodyPattern;
    }

    // Simple placeholder substitution; swap for a real template engine if needed.
    Message render(Map<String, String> vars) {
        return new Message(fill(subjectPattern, vars), fill(bodyPattern, vars));
    }

    private String fill(String pattern, Map<String, String> vars) {
        String out = pattern;
        for (Map.Entry<String, String> e : vars.entrySet()) {
            out = out.replace "{" + e.getKey() + " ", e.getValue());
        }
        return out;
    }
}

class NotificationPreference {
    private final Set<ChannelType> optedIn;
    NotificationPreference(Set<ChannelType> optedIn) { this.optedIn = optedIn; }
    boolean isSubscribed(ChannelType type) { return optedIn.contains(type); }
}

class User {
    private final String id;
    private final String email;
    private final String phone;
    private final String deviceToken;
    private final NotificationPreference preference;

    User(String id, String email, String phone, String deviceToken,
        NotificationPreference preference) {
        this.id = id; this.email = email; this.phone = phone;
        this.deviceToken = deviceToken; this.preference = preference;
    }

    String getId() { return id; }
    NotificationPreference getPreference() { return preference; }
    // Address to hand to a channel, keyed by channel type.
    String addressFor(ChannelType type) {
        switch (type) {
            case EMAIL: return email;
            case SMS:    return phone;
            case PUSH:   return deviceToken;
            default: throw new IllegalStateException("Unknown channel " + type);
        }
    }
}
```

```
}  
}
```

Observer for delivery status (audit/UI hook)

```
interface DeliveryListener {  
    void onResult(String userId, ChannelType type, boolean success, String detail);  
}
```

The service — async dispatch, retries, one failure isolated from others

```
class NotificationService {
    private final ChannelFactory factory = new ChannelFactory();
    private final ExecutorService pool = Executors.newFixedThreadPool(8);
    private final List<DeliveryListener> listeners = new CopyOnWriteArrayList<>();
    private static final int MAX_ATTEMPTS = 3;

    void subscribe(DeliveryListener l) { listeners.add(l); }    // Observer registration

    // Fan out one rendered message to every channel the user opted into.
    // Each channel send runs on its own task -> a slow SMS gateway can't
    // delay email/push, and one channel's exception can't sink the others.
    void notify(User user, NotificationTemplate template, Map<String, String> vars) {
        Message message = template.render(vars);
        for (ChannelType type : ChannelType.values()) {
            if (!user.getPreference().isSubscribed(type)) continue;
            pool.submit(() -> sendWithRetry(user, type, message));
        }
    }

    // Retry with simple backoff; failure on this channel is contained here
    // and reported via the Observer, never thrown back into the caller.
    private void sendWithRetry(User user, ChannelType type, Message message) {
        NotificationChannel channel = factory.get(type);
        String address = user.addressFor(type);
        int attempt = 0;
        while (true) {
            attempt++;
            try {
                channel.send(address, message);
                notifyListeners(user, type, true, "delivered on attempt " + attempt);
                return;
            } catch (NotificationException e) {
                if (attempt >= MAX_ATTEMPTS) {
                    // Give up; in production this is where you'd push to a DLQ
                    // for later inspection/replay instead of dropping it.
                    notifyListeners(user, type, false, "failed after " + attempt + ": " + e.g
                    return;
                }
                sleepBackoff(attempt);    // e.g. 200ms, 400ms, 800ms
            }
        }
    }

    private void sleepBackoff(int attempt) {
        try { Thread.sleep(200L * (1L << (attempt - 1))); }
        catch (InterruptedException ie) { Thread.currentThread().interrupt(); }
    }

    private void notifyListeners(User user, ChannelType type, boolean ok, String detail) {
        for (DeliveryListener l : listeners) l.onResult(user.getId(), type, ok, detail);
    }
}
```

Why a thread pool instead of sending inline? `notify` returns as soon as tasks are queued; the caller (e.g. an order-shipped event handler) isn't stuck waiting on a network round-trip to three different

providers. Bounding the pool (`newFixedThreadPool(8)`) also protects the service from an unbounded burst of notifications spawning unlimited threads.

Why retry inside `sendWithRetry` rather than in `NotificationChannel` ? Retry policy (attempts, backoff) is a dispatch concern, not a channel concern — keeping it in the service means every channel gets the same policy for free, and a channel implementation only needs to know how to send once and how to signal failure.

Use-case walkthrough

```
1. User u = new User("u1", "a@x.com", "+91...", "tok123",
                    new NotificationPreference(Set.of(EMAIL, PUSH)));
2. NotificationTemplate t = new NotificationTemplate("shipped",
    "Order {orderId} shipped", "Your order {orderId} is on its way.");
3. service.notify(u, t, Map.of("orderId", "0-42"));
    -> renders Message once
    -> SMS skipped (not subscribed)
    -> EMAIL task submitted to pool -> EmailChannel.send() succeeds -> listener notified
    -> PUSH task submitted to pool -> PushChannel.send() throws twice,
        succeeds on 3rd attempt after backoff -> listener notified
4. Both tasks ran concurrently; a slow push send never delayed the email send.
```

Extensions an interviewer may probe

- **Durable delivery / at-least-once guarantee** → replace the in-memory executor with a message queue (e.g. SQS/Kafka) and workers that pull, send, and ack; failed messages land in a DLQ instead of being dropped after `MAX_ATTEMPTS` .
 - **Rate limiting per user/channel** → wrap each `NotificationChannel` with a decorator that enforces a quota before calling the real `send` .
 - **New channel (e.g. WhatsApp)** → add a class implementing `NotificationChannel` , register it in `ChannelFactory` ; `NotificationService` and `User` need no changes (Open/Closed).
-

12. Pluggable Cache (LRU / LFU)

The problem in one line. Design an in-memory key-value cache with a fixed capacity whose eviction policy (LRU, LFU, ...) can be swapped at construction time without changing the cache's own code.

Step 1 — Clarify requirements

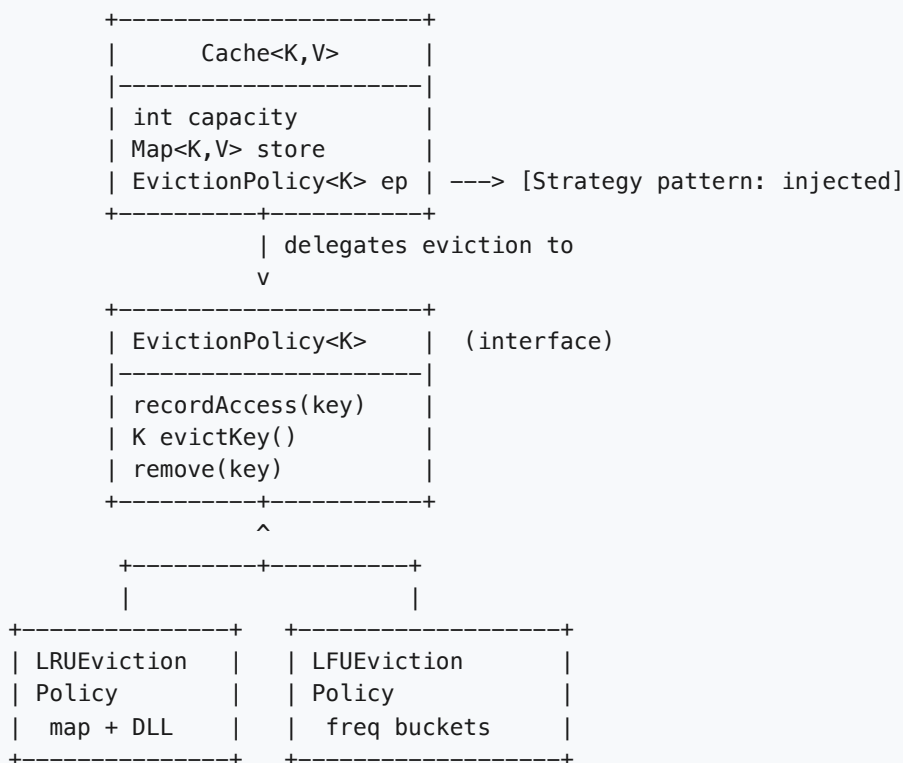
Questions worth asking out loud (and the assumptions this design makes):

- **Which eviction policies?** Least Recently Used and Least Frequently Used, at minimum. *This guide's earlier volume built a standalone LRU cache; here the point is a `Cache` that doesn't know or care which policy it's running* — that's the interesting part.
- **Operations?** `get(key)` and `put(key, value)`, both $O(1)$ regardless of policy.
- **Eviction trigger?** When `put` would exceed capacity, evict exactly one entry first.
- **Concurrency?** Multiple threads read and write the same cache instance concurrently; no entry may be lost or double-counted.
- **Expiry?** Not in the core design — called out as an extension (TTL per entry).
- **Capacity?** Fixed at construction, $\text{capacity} > 0$. Zero/negative capacity is a bug.

Step 2 — Entities (the nouns)

`Cache<K,V>`, `EvictionPolicy<K>` (+ `LRUEvictionPolicy`, `LFUEvictionPolicy`), `Node` (the LRU policy's internal doubly-linked-list entry).

Step 3 — Relationships (class diagram)



Step 4 — Patterns chosen (and why)

- **Strategy** for `EvictionPolicy` — the whole design point. `Cache` never asks "am I LRU or LFU?"; it calls `recordAccess` on every touch and `evictKey` when full. Swapping LRU for LFU (or adding LRU-2, ARC, ...) means writing one new class, zero changes to `Cache`.
- **Composition over inheritance** — `Cache` *has an* `EvictionPolicy`; it does not extend one. This is what makes the policy pluggable at runtime, not just at compile time.
- **Encapsulation of O(1) bookkeeping** — each policy hides its own data structures (a DLL for LRU, frequency buckets for LFU) behind two narrow methods, so `Cache` stays tiny.

Step 5 — Skeleton + key methods

The EvictionPolicy contract

```
interface EvictionPolicy<K> {
    void recordAccess(K key);    // called on every get() hit and every put()
    void add(K key);            // called when a brand-new key is inserted
    void remove(K key);         // called when a key is explicitly removed/overwritten
    K evictKey();               // returns the key to evict; does not remove it itself
}
```

`Cache` owns the actual `key -> value` map; the policy only owns *ordering/frequency* metadata and hands back a key to evict. Keeping value storage out of the policy is what lets the same policy implementation serve caches of any value type `V`.

LRUEvictionPolicy — HashMap + doubly linked list, O(1)

(The mechanics here mirror the standalone LRU Cache built earlier in this guide — same map + DLL trick — just repackaged behind the `EvictionPolicy` interface instead of being baked into the cache itself.)

```
class LRUEvictionPolicy<K> implements EvictionPolicy<K> {
    private final Map<K, Node<K>> nodes = new HashMap<>();
    private final Node<K> head = new Node<>(null); // most-recently-used side
    private final Node<K> tail = new Node<>(null); // least-recently-used side

    LRUEvictionPolicy() { head.next = tail; tail.prev = head; }

    public void add(K key) {
        Node<K> node = new Node<>(key);
        nodes.put(key, node);
        insertAfterHead(node); // new entry is "most recent"
    }

    public void recordAccess(K key) {
        Node<K> node = nodes.get(key);
        if (node == null) return; // defensive: unknown key, no-op
        unlink(node);
        insertAfterHead(node); // move to most-recently-used end
    }

    public K evictKey() {
        if (tail.prev == head) throw new NoSuchElementException("Nothing to evict");
        return tail.prev.key; // least-recently-used = just before t
    }

    public void remove(K key) {
        Node<K> node = nodes.remove(key);
        if (node != null) unlink(node);
    }

    private void insertAfterHead(Node<K> node) {
        node.next = head.next; node.prev = head;
        head.next.prev = node; head.next = node;
    }

    private void unlink(Node<K> node) {
        node.prev.next = node.next; node.next.prev = node.prev;
    }

    private static class Node<K> {
        K key; Node<K> prev, next;
        Node(K key) { this.key = key; }
    }
}
```

LFUEvictionPolicy — frequency buckets, O(1)

Two maps plus a running minimum give O(1) `recordAccess` / `evictKey` without scanning:

- `keyFreq: Map<K, Integer>` — current access count for each key.
- `freqBuckets: Map<Integer, LinkedHashSet<K>>` — all keys currently at a given frequency, in insertion order (so ties break LRU-within-the-same-frequency — the usual LFU rule).

- `minFreq: int` — the smallest frequency currently present, tracked incrementally so `evictKey` never has to search for it.

```
class LFUEvictionPolicy<K> implements EvictionPolicy<K> {
    private final Map<K, Integer> keyFreq = new HashMap<>();
    private final Map<Integer, LinkedHashSet<K>> freqBuckets = new HashMap<>();
    private int minFreq = 0;

    public void add(K key) {
        keyFreq.put(key, 1);
        freqBuckets.computeIfAbsent(1, f -> new LinkedHashSet<>()).add(key);
        minFreq = 1; // a fresh key is always the new min
    }

    public void recordAccess(K key) {
        Integer freq = keyFreq.get(key);
        if (freq == null) return; // defensive: unknown key
        freqBuckets.get(freq).remove(key);
        if (freqBuckets.get(freq).isEmpty() && freq == minFreq) {
            minFreq++; // that bucket just emptied out
        }
        int newFreq = freq + 1;
        keyFreq.put(key, newFreq);
        freqBuckets.computeIfAbsent(newFreq, f -> new LinkedHashSet<>()).add(key);
    }

    public K evictKey() {
        LinkedHashSet<K> bucket = freqBuckets.get(minFreq);
        if (bucket == null || bucket.isEmpty()) {
            throw new NoSuchElementException("Nothing to evict");
        }
        return bucket.iterator().next(); // oldest key at the lowest frequency
    }

    public void remove(K key) {
        Integer freq = keyFreq.remove(key);
        if (freq != null) freqBuckets.get(freq).remove(key);
    }
}
```

Cache — delegates everything to the injected policy

```
class Cache<K, V> {
    private final int capacity;
    private final Map<K, V> store = new HashMap<>();
    private final EvictionPolicy<K> policy; // Strategy: injected, not hardcoded
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();

    Cache(int capacity, EvictionPolicy<K> policy) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity must be > 0");
        if (policy == null) throw new IllegalArgumentException("policy must not be null");
        this.capacity = capacity;
        this.policy = policy;
    }

    // Read path: a plain lookup plus a policy bookkeeping update.
    // Uses the write lock, not the read lock -- recordAccess mutates the
    // policy's internal structures (DLL / frequency buckets), so treating
    // this as "read-only" would corrupt state under concurrent gets.
    public V get(K key) {
        if (key == null) throw new IllegalArgumentException("key must not be null");
        lock.writeLock().lock();
        try {
            if (!store.containsKey(key)) return null; // miss: no bookkeeping to update
            policy.recordAccess(key);
            return store.get(key);
        } finally {
            lock.writeLock().unlock();
        }
    }

    // Write path: insert/overwrite, evicting first if at capacity.
    public void put(K key, V value) {
        if (key == null) throw new IllegalArgumentException("key must not be null");
        lock.writeLock().lock();
        try {
            if (store.containsKey(key)) {
                store.put(key, value);
                policy.recordAccess(key); // overwrite counts as an access
                return;
            }
            if (store.size() >= capacity) {
                K victim = policy.evictKey();
                store.remove(victim);
                policy.remove(victim);
            }
            store.put(key, value);
            policy.add(key);
        } finally {
            lock.writeLock().unlock();
        }
    }

    public int size() {
        lock.readLock().lock(); // pure read, no policy mutation
        try { return store.size(); } finally { lock.readLock().unlock(); }
    }
}
```

Why not a read lock for `get` ? A `ReadWriteLock` only pays off when the "read" path truly doesn't mutate shared state. Here `get` always calls `recordAccess`, which rewires the LRU list or bumps an LFU frequency bucket — that's a write to the policy's internal structures even though the cache's *public* contract looks like a read. Locking it as a write avoids a subtle race where two "readers" both try to unlink/relink the same DLL node. If read traffic dominates and this becomes a bottleneck, the usual follow-up is a lock per shard (partition the keyspace) rather than pretending `get` is lock-free.

Use-case walkthrough

```
Cache<String,String> c = new Cache<>(2, new LRUEvictionPolicy<>());
1. c.put("a", "1")    -> store={a:1}      policy list: [a]
2. c.put("b", "2")    -> store={a:1,b:2}  policy list: [b,a] (b most recent)
3. c.get("a")         -> hit, returns "1"  policy list: [a,b] (a moved to front)
4. c.put("c", "3")    -> at capacity(2): evictKey() -> "b" (tail)
                        store={a:1,c:3}  policy list: [c,a]
5. c.get("b")         -> miss, returns null (b was just evicted)
```

Swap in `LFUEvictionPolicy` with the same Cache code, same call sequence:
step 4 would instead evict whichever of {a,b} has the lower access count --
Cache's `put()/get()` bodies above do not change at all.

Extensions an interviewer may probe

- **TTL / expiry** → add `expiresAt` alongside each value; a background sweeper thread (or a check on every `get`) removes expired entries before consulting the policy at all.
 - **Distributed cache** → single-process locking stops working; this becomes a discussion of consistent hashing across nodes and a cache-invalidation protocol, not this design.
 - **Write-through / write-back to a backing store** → `put` becomes async or delegates to a pluggable `CacheLoader`, another Strategy slot alongside `EvictionPolicy`.
-

13. Ride Hailing / Food Delivery (mini)

The problem in one line. Design a simplified Uber/Ola: a rider requests a trip, the system matches an available driver nearby, tracks the trip through its lifecycle, and computes a fare.

Step 1 — Clarify requirements

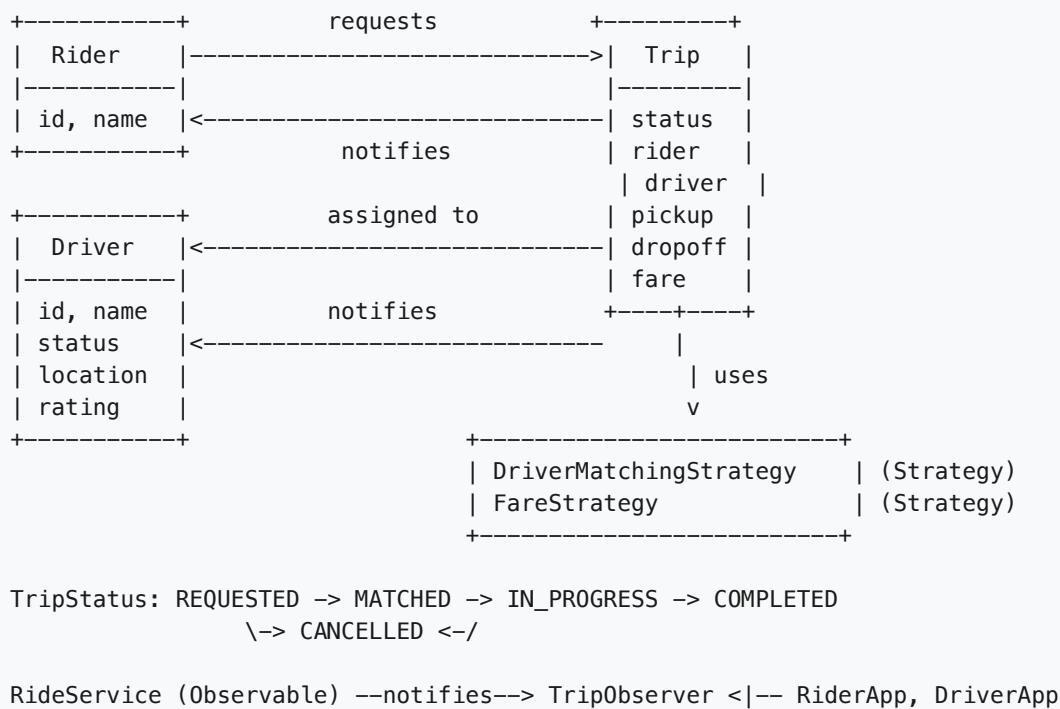
Questions worth asking out loud (and the assumptions this design makes):

- **What triggers matching?** A `Rider` requests a trip from a pickup `Location`; the system must find one `Driver` who is currently `AVAILABLE` and near that location.
- **Matching policy?** Nearest driver by default, but the interviewer may ask for "highest-rated driver" too → pluggable matching strategy.
- **Pricing policy?** Base fare + per-km + surge multiplier. Must be swappable without touching the matching or trip code.
- **Trip lifecycle?** `REQUESTED` → `MATCHED` → `IN_PROGRESS` → `COMPLETED`, with `CANCELLED` reachable from `REQUESTED` or `MATCHED`. No skipping states.
- **Notifications?** Rider and driver apps must be told whenever trip status changes (push notification / SMS in a real system — here, an in-memory listener).
- **Concurrency?** Two riders could request a trip near the same driver at the same instant — that driver must end up matched to **exactly one** of them.
- **Scale (out of scope for the code, but mention it)?** Finding "nearby" drivers by scanning every driver doesn't scale; real systems index drivers with a **geohash or quadtree** and query a cell/radius. This design keeps the model simple (linear scan by distance) and calls out the swap point.
- **Food delivery?** Same skeleton: `Rider` → `Customer`, `Trip` → `Order`, pickup location → restaurant, drop-off → customer address, `Driver` → `Courier`. Matching and status-machine code is identical, so it isn't re-modeled separately here.

Step 2 — Entities (the nouns)

`Rider`, `Driver`, `Location`, `Trip`, `TripStatus`, `DriverMatchingStrategy`, `FareStrategy`, `TripObserver`.

Step 3 — Relationships (class diagram)



Step 4 — Patterns chosen (and why)

- **Strategy** for `DriverMatchingStrategy` — nearest-driver today, highest-rated or ML-ranked tomorrow, without touching `RideService`.
- **Strategy** for `FareStrategy` — base + distance today, surge pricing during peak hours, promo discounts later — same interface, new implementation.
- **Observer** for trip status changes — rider app and driver app both subscribe to a `Trip`'s lifecycle instead of `RideService` calling each of them directly. New subscribers (e.g. an analytics logger) plug in without code changes.
- **State-machine-by-enum** for `TripStatus` — transitions are validated centrally in `Trip.transitionTo`, so an invalid jump (e.g. REQUESTED → COMPLETED) fails loudly.
- **Composition** everywhere: a `Trip` has a rider, a driver, and locations; it does not inherit from them.

Step 5 — Skeleton + key methods

Core value types

```
enum TripStatus { REQUESTED, MATCHED, IN_PROGRESS, COMPLETED, CANCELLED }
enum DriverStatus { AVAILABLE, BUSY, OFFLINE }

class Location {
    final double lat, lon;
    Location(double lat, double lon) { this.lat = lat; this.lon = lon; }

    // Haversine-lite (flat-earth approx is fine for LLD purposes).
    double distanceTo(Location other) {
        double dLat = lat - other.lat, dLon = lon - other.lon;
        return Math.sqrt(dLat * dLat + dLon * dLon);
    }
}
```

Rider and Driver

```
class Rider {
    private final String id;
    private final String name;
    Rider(String id, String name) { this.id = id; this.name = name; }
    String getId() { return id; }
}

class Driver {
    private final String id;
    private final String name;
    private volatile DriverStatus status = DriverStatus.OFFLINE;
    private Location location;
    private double rating = 5.0;

    Driver(String id, String name) { this.id = id; this.name = name; }

    String getId() { return id; }
    Location getLocation() { return location; }
    void updateLocation(Location loc) { this.location = loc; }
    DriverStatus getStatus() { return status; }
    double getRating() { return rating; }

    // Atomic check-and-flip: only succeeds if the driver was AVAILABLE.
    // This is the critical section that prevents double-booking (see Step 5, RideService).
    synchronized boolean tryLock() {
        if (status != DriverStatus.AVAILABLE) return false;
        status = DriverStatus.BUSY;
        return true;
    }

    synchronized void release() { status = DriverStatus.AVAILABLE; }
}
```

Matching strategy (Strategy pattern)

```
interface DriverMatchingStrategy {
    // Returns the chosen driver, or null if none available.
    Driver select(List<Driver> candidates, Location pickup);
}

class NearestDriverStrategy implements DriverMatchingStrategy {
    public Driver select(List<Driver> candidates, Location pickup) {
        return candidates.stream()
            .filter(d -> d.getStatus() == DriverStatus.AVAILABLE)
            .min(Comparator.comparingDouble(d -> d.getLocation().distanceTo(pickup)))
            .orElse(null);
    }
}

class HighestRatedDriverStrategy implements DriverMatchingStrategy {
    public Driver select(List<Driver> candidates, Location pickup) {
        return candidates.stream()
            .filter(d -> d.getStatus() == DriverStatus.AVAILABLE)
            .max(Comparator.comparingDouble(Driver::getRating))
            .orElse(null);
    }
}
```

Fare strategy (Strategy pattern)

```
interface FareStrategy {
    double calculate(Location pickup, Location dropoff);
}

class BaseDistanceFareStrategy implements FareStrategy {
    private final double baseFare, perKm, surgeMultiplier;

    BaseDistanceFareStrategy(double baseFare, double perKm, double surgeMultiplier) {
        this.baseFare = baseFare; this.perKm = perKm; this.surgeMultiplier = surgeMultiplier;
    }

    public double calculate(Location pickup, Location dropoff) {
        double km = pickup.distanceTo(dropoff);
        return (baseFare + km * perKm) * surgeMultiplier;
    }
}
```

Observer for trip-status notifications

```
interface TripObserver {
    void onStatusChanged(Trip trip, TripStatus oldStatus, TripStatus newStatus);
}

class RiderAppNotifier implements TripObserver {
    public void onStatusChanged(Trip trip, TripStatus oldStatus, TripStatus newStatus) {
        System.out.println("[Rider " + trip.getRider().getId() + "] trip is now " + newStatus);
    }
}

class DriverAppNotifier implements TripObserver {
    public void onStatusChanged(Trip trip, TripStatus oldStatus, TripStatus newStatus) {
        if (trip.getDriver() != null) {
            System.out.println("[Driver " + trip.getDriver().getId() + "] trip is now " + newStatus);
        }
    }
}
```


Trip — the state machine

```
class Trip {
    private static final Map<TripStatus, Set<TripStatus>> ALLOWED = Map.of(
        TripStatus.REQUESTED,    Set.of(TripStatus.MATCHED, TripStatus.CANCELLED),
        TripStatus.MATCHED,      Set.of(TripStatus.IN_PROGRESS, TripStatus.CANCELLED),
        TripStatus.IN_PROGRESS,  Set.of(TripStatus.COMPLETED),
        TripStatus.COMPLETED,    Set.of(),
        TripStatus.CANCELLED,     Set.of()
    );

    private final String id;
    private final Rider rider;
    private final Location pickup, dropoff;
    private Driver driver;
    private TripStatus status = TripStatus.REQUESTED;
    private double fare;
    private final List<TripObserver> observers;

    Trip(String id, Rider rider, Location pickup, Location dropoff, List<TripObserver> observ
        this.id = id; this.rider = rider; this.pickup = pickup; this.dropoff = dropoff;
        this.observers = observers;
    }

    Rider getRider() { return rider; }
    Driver getDriver() { return driver; }
    Location getPickup() { return pickup; }
    Location getDropoff() { return dropoff; }
    TripStatus getStatus() { return status; }
    void assignDriver(Driver d) { this.driver = d; }
    void setFare(double fare) { this.fare = fare; }

    // Centralized, validated transition. Anything not in ALLOWED throws.
    void transitionTo(TripStatus next) {
        Set<TripStatus> allowed = ALLOWED.get(status);
        if (allowed == null || !allowed.contains(next)) {
            throw new IllegalStateException("Cannot go from " + status + " to " + next);
        }
        TripStatus old = status;
        status = next;
        for (TripObserver obs : observers) obs.onStatusChanged(this, old, next);
    }
}
```

RideService — matching, with concurrency and error handling

```
class RideService {
    private final List<Driver> drivers = new ArrayList<>();
    private final Map<String, Trip> trips = new ConcurrentHashMap<>();
    private final List<TripObserver> observers = List.of(new RiderAppNotifier(), new DriverAppNotifier());
    private DriverMatchingStrategy matchingStrategy = new NearestDriverStrategy();
    private FareStrategy fareStrategy = new BaseDistanceFareStrategy(2.5, 1.2, 1.0);
    private final AtomicLong tripSeq = new AtomicLong(0);

    void registerDriver(Driver d) { drivers.add(d); }
    void setMatchingStrategy(DriverMatchingStrategy s) { this.matchingStrategy = s; }
    void setFareStrategy(FareStrategy s) { this.fareStrategy = s; }

    // Request a trip: pick a driver and lock it atomically.
    // synchronized so two riders requesting at the same instant can't both
    // "see" the same driver as available and both proceed to match it –
    // the classic double-booking race on shared driver state.
    public synchronized Trip requestTrip(Rider rider, Location pickup, Location dropoff) {
        Trip trip = new Trip("TR" + tripSeq.incrementAndGet(), rider, pickup, dropoff, observers);
        trips.put(trip.getId(), trip);

        Driver chosen = matchingStrategy.select(drivers, pickup);
        if (chosen == null) {
            throw new NoSuchElementException("No drivers available near pickup");
        }

        // tryLock() is the real guard: even under this method's own lock, it re-checks
        // status with its own synchronized block, so it's safe if callers ever bypass
        // requestTrip's lock (e.g. a driver going OFFLINE concurrently).
        if (!chosen.tryLock()) {
            throw new IllegalStateException("Driver " + chosen.getId() + " was already taken");
        }

        trip.assignDriver(chosen);
        trip.transitionTo(TripStatus.MATCHED);
        return trip;
    }

    public void startTrip(String tripId) {
        Trip trip = getTripOrThrow(tripId);
        trip.transitionTo(TripStatus.IN_PROGRESS); // throws if not MATCHED
    }

    // Complete a trip: compute fare, release the driver back to the pool.
    public double completeTrip(String tripId) {
        Trip trip = getTripOrThrow(tripId);
        double fare = fareStrategy.calculate(trip.getPickup(), trip.getDropoff());
        trip.setFare(fare);
        trip.transitionTo(TripStatus.COMPLETED); // throws if not IN_PROGRESS
        trip.getDriver().release();
        return fare;
    }

    public void cancelTrip(String tripId) {
        Trip trip = getTripOrThrow(tripId);
        trip.transitionTo(TripStatus.CANCELLED); // throws if already IN_PROGRESS/COMPLETED
        if (trip.getDriver() != null) trip.getDriver().release();
    }
}
```

```

private Trip getTripOrThrow(String tripId) {
    Trip trip = trips.get(tripId);
    if (trip == null) throw new NoSuchElementException("Unknown trip: " + tripId);
    return trip;
}
}

```

Why `synchronized` on `requestTrip`, plus a `synchronized tryLock()` on `Driver`? This is a defense-in-depth, two-layer lock:

1. `requestTrip` is serialized so the "pick a driver" decision itself isn't racing against another `requestTrip` call — otherwise `matchingStrategy.select(...)` could return the *same* driver to two concurrent requests before either has locked it.
2. `Driver.tryLock()` does its own atomic check-and-flip (`status == AVAILABLE` then set `BUSY`) inside a `synchronized` block on the driver itself. This is the actual correctness guarantee — even if the outer lock were removed or a second `RideService` instance existed, no two threads can flip the same driver from `AVAILABLE` to `BUSY` at once. One of them will see `status != AVAILABLE` and get `false` back, and `requestTrip` turns that into an `IllegalStateException` instead of silently double-booking.

At real scale, replace the linear scan in `NearestDriverStrategy` with a query against a **geohash or quadtree index** keyed by driver location, refreshed as drivers move; the `DriverMatchingStrategy` interface doesn't change, only what's inside `select`.

Use-case walkthrough

```

1. rideService.registerDriver(driver1); // driver1.status = AVAILABLE, near rider
2. Trip trip = rideService.requestTrip(rider, pickup, dropoff);
   -> NearestDriverStrategy scans AVAILABLE drivers, picks driver1
   -> driver1.tryLock() flips AVAILABLE -> BUSY
   -> trip.transitionTo(MATCHED) -> observers notify rider app + driver app
3. rideService.startTrip(trip.getId());
   -> transitionTo(IN_PROGRESS); REQUESTED->IN_PROGRESS directly would throw
4. double fare = rideService.completeTrip(trip.getId());
   -> BaseDistanceFareStrategy computes base + distance*rate
   -> transitionTo(COMPLETED); driver1.release() -> BUSY -> AVAILABLE
5. A second rider requests a trip at the same instant driver1 was BUSY:
   -> tryLock() returns false for driver1 (already taken)
   -> requestTrip throws IllegalStateException; matching falls back / retries
       with the next-nearest AVAILABLE driver.

```

Extensions an interviewer may probe

- **Surge pricing by geo-cell** → new `FareStrategy` that looks up a demand multiplier per zone; `RideService` code is untouched (Open/Closed).
- **Driver rejects a match / times out** → add a `PENDING_DRIVER_ACCEPT` sub-state before `MATCHED`, with a timeout that releases the driver and re-matches.
- **Food delivery variant** → swap `Rider / Trip` for `Customer / Order`, add a `RestaurantPrepTime` step before pickup; matching and fare strategies carry over unchanged, showing the pattern choice was the right abstraction.

